



UNIVERSIDAD NACIONAL
DE EDUCACIÓN A DISTANCIA

Escuela Técnica Superior de Ingeniería Informática

[TÍTULO DEL TRABAJO]

Gabriel Hernández Vallejo

Director/a: [Director/a]

Co-director/a: [Co-director/a]

Trabajo de Fin de Máster

Máster Universitario
en Ingeniería Informática

[Fecha de presentación/Convocatoria]

Agradecimientos

[Quisiera dedicar ...]

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Palabras clave: palabra1, palabra2, palabra3

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Keywords: keyword1, keyword2, keyword3

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Propuesta y objetivos	2
1.3. Estructura del documento	2
2. Estado del Arte	3
2.1. Soluciones de escritorio	3
2.2. Soluciones móviles	4
2.3. Soluciones con realidad aumentada	5
3. Materiales y métodos	7
3.1. Diseño metodológico del proyecto	7
3.2. Entorno de trabajo	7
3.3. Materiales empleados	9
3.4. Análisis de requisitos	9
3.4.1. Requisitos funcionales	9
3.4.2. Requisitos no funcionales	11
3.4.3. Casos de uso	12
3.5. Diseño del sistema	19
3.6. Implementación del sistema	19
3.7. Validación y pruebas	24
3.7.1. Pruebas del CU-01: Configuración conexión ROS	24
3.7.2. Pruebas del CU-02: Configuración idioma	25
3.7.3. Pruebas del CU-03: Asignación de <i>frame</i> a <i>AprilTag</i>	25
3.7.4. Pruebas del CU-04: Configuración de nube de puntos	25
3.7.5. Pruebas del CU-05: Configuración visualización ruta planificada	25
3.7.6. Pruebas del CU-06: Visualizar/Ocultar <i>frames</i>	26
3.7.7. Pruebas del CU-07: Teleoperación	26
3.7.8. Pruebas del CU-08: Planificar ruta	27
3.7.9. Pruebas del CU-09: Medición entre <i>frames</i>	27

3.7.10. Pruebas del CU-10: Ajuste de posición de <i>frames</i> asignados a marcas visuales	28
3.7.11. Pruebas del CU-11: Ajuste de escala de <i>frames</i> asignados a marcas visuales	28
3.7.12. Pruebas del CU-12: Suscripción a <i>topics</i>	29
3.7.13. Pruebas del CU-13: Publicación en <i>topics</i>	29
3.7.14. Pruebas del CU-14: Modificación de parámetros en nodos	29
3.8. Consideraciones éticas y legales	30
4. Resultados	31
4.1. Software desarrollado	31
4.2. Validación de los requisitos	39
4.3. Resultados de las pruebas	40
4.4. Rendimiento	42
4.5. Limitaciones encontradas	42
4.6. Resumen de resultados	43
5. Conclusiones y trabajos futuros	45
5.1. Conclusiones	45
5.2. Líneas de trabajo futuras	45
Bibliografía y referencias	49
A. Ejemplos en L^AT_EX	51
A.1. Referencias, citas y bibliografía	51
A.1.1. Referencias	51
A.1.2. Bibliografía	51
A.2. Figuras	52
A.3. Tablas	52
A.4. Código	53
A.5. Listas	54
A.6. Ecuaciones	54

Índice de figuras

3.1. Módulo TF	20
3.2. Módulo PointClouds	21
3.3. Módulo SpatialControl	22
3.4. Módulo ROSSubscriber	22
3.5. Módulo ROSService	23
4.1. Visualización <i>transforms</i>	31
4.2. Visualización velocidad lineal	32
4.3. Visualización velocidad angular	33
4.4. Ocultación de frames	33
4.5. Visualización de frames	34
4.6. Teleoperación	34
4.7. Planificación de ruta	35
4.8. Medición de frames	35
4.9. Ajuste de escala de frames	36
4.10. Configuración de nube de puntos	36
4.11. Visualización de nube de puntos	37
4.12. Suscripción a <i>topics</i>	37
4.13. Publicación en <i>topic</i>	38
4.14. Configuración de parámetros	38
4.15. Configuración	39
A.1. Ejemplo de figura	52

Índice de tablas

3.3. Casos de prueba de la configuración de conexión con ROS	24
3.4. Casos de prueba de la configuración de idioma	25
3.5. Casos de prueba de la asignación de <i>frame</i> a <i>AprilTag</i>	25
3.6. Casos de prueba de la configuración de nube de puntos	25
3.7. Casos de prueba de configuración de visualización de ruta planificada	26
3.8. Casos de prueba de visualización/ocultación de <i>frames</i>	26
3.9. Casos de prueba de teleoperación	27
3.10. Casos de prueba de planificación de ruta	27
3.11. Casos de prueba de medición entre <i>frames</i>	28
3.12. Casos de prueba del ajuste de la posición de <i>frames</i> asignados a marcas visuales	28
3.13. Casos de prueba del ajuste de escala de <i>frames</i> asignados a marcas visuales .	29
3.14. Casos de prueba de la suscripción a <i>topics</i>	29
3.15. Casos de prueba de la publicación en <i>topics</i>	29
3.16. Casos de prueba de la modificación de parámetros	29
4.1. Estado de los requisitos funcionales y no funcionales	40
4.2. Resultados de las pruebas de integración	41
A.1. Ejemplo de tabla	53

Capítulo 1

Introducción

1.1. Motivación

ROS2 (*Robot Operating System 2*) es un conjunto de herramientas de *software* que proporcionan una plataforma estandarizada para desarrollar aplicaciones de robótica (*ROS: Why ROS?* — *ros.org* s.f.).

Esta tecnología se caracteriza por ser una solución multidominio (permite desarrollar aplicaciones para robots externos e internos, industriales o domésticos, terrestres, aéreos o marítimos, etc.) y multiplataforma (es soportado por diferentes sistemas operativos Linux, Windows y MacOS además de otras plataformas embebidas). Por otro lado, se trata de un conjunto de herramientas de código abierto (*open source*) distribuida a través de licencias permisivas (como Apache 2) que facilitan la distribución de soluciones personalizadas propietarias (*ROS: Why ROS?* — *ros.org* s.f.).

La adopción de las soluciones proporcionadas por ROS2 en el año 2023 experimentó un incremento del 22,9% de compañías (Katherine Scott s.f.).

La interfaz proporcionada por ROS2 permite desarrollar nuevos métodos de interacción humano-robot de forma que sea posible visualizar diferentes parámetros de la situación actual del sistema o teleoperar los diferentes dispositivos desplegados. Una de las maneras más habituales de realizar las operaciones anteriormente mencionadas es empleando un computador personal (PC) ya sea o no portátil, lo que limita la movilidad y maniobrabilidad del operador del robot.

Existen tendencias más actuales que emplean dispositivos montados en la cabeza (HMD; *Head Mounted Displays*) como *Microsoft HoloLens*, *MetaQuest* o *HTC Vive* que permiten mezclar el mundo real y el virtual mediante la realidad aumentada empleando interfaces más intuitivas, directas y que permiten la movilidad total o parcial del operador. Esto facilita el aprendizaje, la operativa y la solución de errores a la vez que reduce la carga cognitiva necesaria con respecto a la utilización de interfaces gráficas bidimensionales no contextualizadas (Richards et al. 2025; Calderón-Sesmero et al. 2025). Sin embargo, estos dispositivos conti-

nuan teniendo un coste demasiado elevado para muchos usuarios, lo que limita la capacidad de despliegue de las soluciones desarrolladas a las organizaciones más pudientes. Además, requieren distribuir el centro operativo en diferentes instrumentos en lugar de centralizar las acciones y datos en un único dispositivo como los *smartphones* o *tablets* que cumplen múltiples propósitos.

A pesar de las aparentes ventajas económicas y operacionales de emplear los dispositivos móviles como interfaz en la visualización de datos o tele-operación de robots, no existen demasiadas soluciones integradas que permitan realizar dichas acciones empleando realidad aumentada.

1.2. Propuesta y objetivos

El proyecto propone desarrollar una interfaz multiplataforma (Android/iOS) que permita la interacción con robots (visualización de datos y tele-operación) basados en ROS2 mediante realidad aumentada eliminando las barreras materiales y económicas existentes en la teleoperación y visualización de datos.

Para ello, se deben evaluar las soluciones existentes y su compatibilidad con ROS2 y la realidad aumentada móvil para adoptarlas y adaptarlas en caso de ser necesario. También es necesario crear un sistema de comunicaciones ligero y multiplataforma coordinado preferiblemente desde el dispositivo móvil eliminando la necesidad de agentes coordinadores externos.

Para proporcionar la visualización de datos y la teleoperación es preciso incorporar interfaces en realidad aumentada intuitivas de manera que los operadores puedan contextualizar en todo momento la situación del sistema y tomar las medidas necesarias en el desarrollo de su trabajo (i.e. cambiar parámetros de los agentes).

Es necesario, además, realizar una evaluación de la usabilidad y la latencia del sistema de manera que se cuente con una fiabilidad suficiente como para poder desplegar la solución en entornos de operación reales.

1.3. Estructura del documento

TBD

Capítulo 2

Estado del Arte

En este apartado se estudiarán las soluciones existentes en teleoperación y visualización de parámetros de robots mediante computador, interfaces móviles y realidad aumentada.

2.1. Soluciones de escritorio

En cuanto a la operación telemática y visualización de parámetros de robots mediante computador existen distintas soluciones que ofrecen diferentes funcionalidades. Entre ellas cabe destacar *RViz*, *Foxglobe Studio* o *Webviz*.

RViz es el visualizador en tres dimensiones (3D) oficial de escritorio para el marco de trabajo (*framework*) ROS (*RViz User Guide* [2014]; *ROS 2 Documentation: Kilted documentation* — *docs.ros.org* s.f.). La aplicación permite cargar un modelo del robot, nubes de puntos, mapas, imágenes, estados del robot, etc. configurar diferentes cámaras en la visualización del gemelo del robot en el computador y realizar distintas acciones como seleccionar elementos en el panel de trabajo, centrar las cámaras a una localización especificada, medir la distancia existente entre dos puntos del espacio, establecer el punto inicial del sistema de localización y el objetivo, etc. Además, permite realizar simulaciones de manera que los operadores puedan validar las acciones previamente a ejecutarlas en el entorno real.

A pesar de que *RViz* puede resultar una herramienta realmente útil para la simulación o teleoperación de sistemas robóticos, la digitalización de los modelos 3D de los robots puede ser compleja para usuarios que no cuenten con conocimiento de modelado por computador o del formato URDF (*Unified Robotics Description Format*), por lo que los resultados finales pueden no ser los más óptimos. Además, el entorno de operación de la aplicación está basado en la recreación del entorno real mediante gemelos digitales, lo que impide la operación directa sobre la robótica instalada y necesita de navegación por diferentes paneles y ventanas para modificar parámetros, lo que puede generar confusiones o excesivas cargas cognitivas para operadores no experimentados o familiarizados con la herramienta.

Foxglobe Studio es una plataforma para registrar, subir, organizar y visualizar datos de

registro multimodales como series de tiempo, registros de texto, vídeo, 3D, mapas y otros (*Foxglove Documentation* — *docs.foxglove.dev* s.f.). La aplicación permite la integración con ROS2 mediante *Foxglobe Bridge*. De manera similar al caso anterior, es posible cargar modelos, audio, mapas, registros de texto, etc. y además de otras operativas, permite la teleoperación básica publicando mensajes en un *topic* específico.

A diferencia de *RViz*, *Foxglobe Studio* no solo permite utilizar otros *framework* como *PX4* o *LeRobot*, sino que se trata de una aplicación más compleja con mayores funcionalidades que permite una gestión de grano fino de los diferentes procesos de la visualización de parámetros y gestión de datos. Sin embargo, se pueden encontrar las mismas problemáticas que con la herramienta anterior, potenciando, en este caso, la complejidad de utilización de la aplicación por estar más enfocada a procesos industriales en el que los operadores cuentan con mayor experiencia, de manera que puede ser inconveniente para usuarios no instruidos adecuadamente.

Webviz permite visualizar parámetros del robot empleando el navegador (*webviz* — *webviz.io* s.f.), lo que permite cruzar la frontera de los dispositivos de mayor potencia como portátiles y computadores de escritorio hacia dispositivos móviles como *smartphones* y *tablets*. Además, se trata de una herramienta modular, de modo que se pueden añadir nuevos componentes que permitan visualizar otros datos diferentes.

A pesar de que esta aplicación permite ser empleada en todos los dispositivos al tratarse de una herramienta web, cuenta con problemáticas similares a las mencionadas anteriormente.

2.2. Soluciones móviles

En cuanto a la operación telemática y visualización de parámetros empleando dispositivos móviles se pueden encontrar soluciones como *ROS-Mobile*, *Robot Steering App*, *ROS2 Connect Mobile* o soluciones propietarias de ciertos fabricantes de robots.

ROS-Mobile es una aplicación Android diseñada para el control dinámico y visualización de sistemas robóticos móviles ROS (*ROS-Mobile - ROS Wiki* — *wiki.ros.org* s.f.) basada en menús para la visualización de parámetros y la operación del robot.

Robot Steering App es una aplicación iOS que permite controlar la velocidad lineal y angular, la gestión de los *topics*, ejecutar paradas de emergencia y obtener flujos de vídeo en tiempo real de las cámaras de los robots ROS2 (*Robot Steering App - App Store* — *apps.apple.com* s.f.).

ROS2 Connect Mobile proporciona una interfaz móvil para el monitoreo de *topics* de ROS2 (*App ROS2 Connect Mobile - App Store* — *apps.apple.com* s.f.). A diferencia de las otras aplicaciones no permite teleoperar los robots.

Las aplicaciones existentes en su mayoría se tratan de soluciones básicas para la visualización y operación en entornos limitadas en la interacción espacial empleando, además, navegación mediante menús, lo que aumenta la carga cognitiva del operador.

2.3. Soluciones con realidad aumentada

Existen diferentes soluciones que emplean realidad aumentada para la visualización de parámetros y teleoperación de robots. Sin embargo, en su mayoría se tratan de herramientas diseñadas para solucionar un problema concreto empleando dispositivos HMD (*Head Mounted Display*) en lugar de tratarse de aplicaciones multidominio que puedan ser aplicadas en diferentes entornos y dispositivos.

Adekoya, Sgorbissa y Recchiuto 2025 introduce HORUS, una interfaz de realidad aumentada para gestionar equipos de robots móviles empleando *Meta Quest 2*.

Regal et al. 2024 introduce AR-STAR una herramienta de realidad aumentada que permite modificar nubes de puntos empleando sobre superficies el visor *HoloLens 2* para la reparación de superficies corroídas mediante robots tele-operados.

Ai, Kazanzides y Azimi 2024 proponen una solución basada en realidad aumentada para la operativa telemática y visualización de datos de robots quirúrgicos empleando *HoloLens 2* para controlar el movimiento de la cabeza, las órdenes verbales y proporcionar la retroalimentación al usuario.

Neves, Serrario y Pires 2018 introduce una implementación de interfaz de realidad mixta mediante *Microsoft HoloLens* con para proporcionar información sobre la tarea que se está realizando e interactuando con objetos virtuales como si se estuviera interactuando con objetos reales.

Yang et al. 2025 propone una demostración de colaboración robot-humano empleando realidad extendida empleando *Meta Quest 3*.

Schmidt et al. 2022 introduce una solución para integrar los robots industriales, la programación industrial *offline* y herramientas de simulación, plataformas de optimización y una interfaz de realidad aumentada empleando *HoloLens* de manera que se permita al operador ser una parte más del sistema de producción robotizado.

Maeda et al. 2024 desarrollan una solución para el control de entornos de construcción mediante sensores y camaras integrados en conos delimitadores y una interfaz de realidad virtual para HMD que permite situar al usuario en el terreno.

Los terminales móviles como *smartphones* y *tablets* también han sido empleados, en menor medida, como interfaz humano-robot.

Friske 2024 propone una solución para proporcionar información del entorno contextualizada en situaciones de emergencia mediante la comunicación con el robot y la visualización de información en la pantalla del terminal.

Natephra y Motamedi 2019 emplea realidad mixta para proporcionar visualización de datos gráficos de modelado de información de construcción (BIM; *Building Information Modeling*) en terminal móvil.

Rosales, Deshpande y Anand 2021 introduce una solución de realidad aumentada para la recolección y visualización de datos en tiempo real en terminal móvil empleando sensores

del internet de las cosas (IoT) en una fábrica de la automovilística Volvo.

Como puede apreciarse, la mayoría de las soluciones móviles que emplean realidad aumentada en dispositivos móviles están centradas en resolver un problema específico. Zea y Hanebeck 2021, sin embargo, introduce una aplicación para la visualización de datos y teleoperación en ROS1 empleando dispositivos móviles (Android/iOS/*Microsoft HoloLens*), lo que permite visualizar los diferentes datos y parámetros de una manera más directa.

Capítulo 3

Materiales y métodos

3.1. Diseño metodológico del proyecto

En el proyecto se empleó el modelo de desarrollo incremental de *software*, el cual permite dividir los esfuerzos en incrementos, de forma que cada uno se integra con los anteriores y se prueba de manera independiente. Se optó por seguir esta metodología puesto que en las primeras etapas se desconocía las tecnologías empleadas más allá de sus fundamentos, por lo que se requería de un proceso de aprendizaje y experimentación que permitiera la familiarización con las herramientas a emplear a la par que se concretaba el diseño final y los requisitos principales que se debían cubrir.

3.2. Entorno de trabajo

Durante el desarrollo del proyecto se empleó un entorno de trabajo basado en linux, específicamente la distribución *Linux Mint 22.2* con el entorno de escritorio *Cinnamon*. Se empleó un equipo con las siguientes características:

- Procesador: AMD Ryzen 7 3700X.
- Memoria RAM: 16 GB DDR4.
- Almacenamiento: 500 GB SSD NVMe 2.0. 2TB HDD.
- Tarjeta gráfica: NVIDIA GeForce RTX 3070.

En lo referente a los dispositivos móviles empleados para la prueba de la aplicación, se emplearon los siguientes:

- **Xiaomi 11T Pro** con sistema operativo Android 14
- **iPhone 15 Plus:** con sistema operativo iOS 26

Debido a la carencia de equipamiento específico, no se emplearon robots físicos, sino que fueron simulados mediante el uso de un *software*, concretamente *Gazebo*, que permite la simulación de robots y entornos en 3D.

Para el desarrollo *software* se emplearon las siguientes herramientas:

- **Visual Studio Code:** IDE para el desarrollo de aplicaciones web y de escritorio, que incluye herramientas de depuración y extensiones para diferentes lenguajes de programación.
- **Git:** Sistema de control de versiones distribuido, que permite llevar un registro de los cambios realizados en el código fuente y colaborar con otros desarrolladores.
- **GitHub:** Plataforma de alojamiento de repositorios Git, que permite compartir el código fuente y colaborar con otros desarrolladores.
- **Unity:** Motor de desarrollo de videojuegos y aplicaciones interactivas, que permite crear aplicaciones para diferentes plataformas, incluyendo Android.
- **Android Studio:** IDE para el desarrollo de aplicaciones Android, que incluye herramientas de depuración y emulación de dispositivos. En concreto, se empleó para las pruebas de contenerización del servicio ROS necesario para el funcionamiento de la aplicación, puesto que permite la instalación de *termux* y la consiguiente ejecución de contenedores en un entorno Android.
- **Unity Hub:** Herramienta de gestión de proyectos y versiones de Unity, que permite crear y administrar proyectos de Unity y descargar diferentes versiones del motor.
- **ROS:** Sistema operativo para robots, que proporciona un conjunto de herramientas y bibliotecas para el desarrollo de aplicaciones robóticas.
- **C#:** Lenguaje de programación orientado a objetos, que se empleó para el desarrollo de la aplicación en Unity.
- **Python:** Lenguaje de programación interpretado, que se empleó para el desarrollo de los nodos ROS.
- **C++:** Lenguaje de programación compilado, que se empleó para el desarrollo del plugin de detección de AprilTags.
- **Termux:** Emulador de terminal para Android, que permite ejecutar aplicaciones de línea de comandos y contenedores en un entorno Android.
- **Gazebo:** Simulador de robots y entornos en 3D, que permite la simulación de robots y entornos en un entorno virtual.

- **Rviz:** Herramienta de visualización de datos de ROS, que permite visualizar los datos generados por los nodos ROS en un entorno gráfico.
- **Terminator:** Emulador de terminal para Linux, que permite ejecutar aplicaciones de línea de comandos en un entorno Linux.
- **Terminal de Linux:** Emulador de terminal para Linux, que permite ejecutar aplicaciones de línea de comandos en un entorno Linux.
- **Draw.io:** Herramienta web que permite crear diagramas UML, de entidad relación, flujogramas, etc.

3.3. Materiales empleados

Para la realización del proyecto se emplearon los siguientes materiales y repositorios:

- **ROS-TCP-Endpoint:** Paquete de ROS que permite la comunicación entre un nodo ROS y una aplicación externa a través de TCP/IP.
- **ROS-TCP-Connector:** Interfaz C# de alto nivel para la comunicación entre la aplicación y los nodos ROS.
- **AprilTags:** Sistema de marcadores visuales que permite la detección y seguimiento de objetos en imágenes y videos.
- **AprilRobotics/apriltag:** Librería C++ que permite la detección y seguimiento de los marcadores visuales (*AprilTags*).

3.4. Análisis de requisitos

A continuación se muestran los requisitos funcionales y no funcionales que deben ser cubiertos en el proyecto.

3.4.1. Requisitos funcionales

ID	Nombre	Descripción	Prioridad
RF-001	Conexión con ROS	El sistema permitirá se conectará al servidor TCP-IP para realizar solicitudes.	Alta
RF-002	Detección de marcas visuales	El sistema deberá reconocer marcas visuales características (<i>AprilTags</i>).	Alta

RF-003	Asignación de <i>frames</i> a marcas visuales	El usuario podrá asignar un <i>frame</i> concreto a una marca visual (<i>AprilTags</i>).	Alta
RF-004	Visualización <i>frames</i>	El sistema permitirá a los usuarios visualizar los <i>frames</i> presentes en el sistema.	Alta
RF-005	Visualización orientación <i>frames</i>	El sistema permitirá conocer la orientación de los <i>frames</i> en todo momento.	Alta
RF-006	Visualización posición <i>frames</i>	El sistema permitirá conocer la posición de los <i>frames</i> en todo momento.	Alta
RF-007	Visualización estado <i>frames</i>	El sistema permitirá conocer el estado de los <i>frames</i> (activo, parado o <i>idle</i>).	Alta
RF-008	Visualización velocidad lineal <i>frames</i>	El sistema permitirá conocer la velocidad lineal de los <i>frames</i> .	Alta
RF-009	Visualización velocidad angular <i>frames</i>	El sistema permitirá conocer la velocidad angular de los <i>frames</i> .	Alta
RF-010	Visualización <i>jitter</i> <i>frames</i>	El sistema permitirá conocer si el <i>frame</i> tiene <i>jitter</i> (fluctuación en tiempo ejecución de instrucciones).	Media
RF-011	Visualización <i>jump</i> <i>frames</i>	El sistema permitirá conocer si el <i>frame</i> se mueve con saltos (<i>jump</i>).	Media
RF-012	Visualización <i>flipping</i> <i>frames</i>	El sistema permitirá conocer si el <i>frame</i> se produce <i>flipping</i> .	Media
RF-013	Ocultar visualización <i>frames</i>	El sistema permitirá ocultar <i>frames</i> .	Alta
RF-014	Ajuste posición <i>frames</i> asignados	El sistema permitirá ajustar la posición de los <i>frames</i> asignados a los <i>AprilTag</i> .	Media
RF-015	Ajuste escala <i>frames</i>	El sistema permitirá ajustar la escala de los <i>frames</i> .	Media
RF-016	Medición <i>frames</i>	El sistema permitirá conocer la distancia que existe entre dos <i>frames</i> .	Baja
RF-017	Configuración <i>frames</i> nube puntos	El usuario podrá asociar a cada <i>frame</i> un <i>topic</i> en el que se emitan mensajes PointCloud2 (nube de puntos).	Baja
RF-018	Visualización nubes de puntos <i>frames</i>	El sistema mostrará las nubes de puntos de sensores en los <i>frames</i> configurados.	Baja

RF-019	Parar ejecución	El sistema permitirá pausar momentáneamente la actualización de la posición de los <i>frames</i> para que puedan realizarse comprobaciones.	Baja
RF-020	Configuración del <i>frame</i> objetivo de ruta planificada	El usuario podrá asignar el <i>topic</i> encargado de la publicación de la ruta planificada al <i>frame</i> raíz.	Media
RF-021	Visualización de ruta planificada	El sistema permitirá conocer la ruta planificada de cada <i>frame</i> .	Media
RF-022	Planificación de ruta	El sistema permitirá crear un plan de ruta para un robot concreto (empleando <i>topic</i> específico).	Media
RF-023	Operación remota (teleoperación)	El sistema permitirá operar a cada robot de forma independiente (empleando <i>topic</i> específico).	Media
RF-024	Suscripción a <i>topics</i>	El usuario podrá suscribirse a <i>topics</i> y observar los contenidos publicados en ellos.	Baja
RF-025	Publicación en <i>topics</i>	El usuario podrá publicar mensajes en <i>topics</i> específicos.	Baja
RF-026	Modificación de parámetros	El usuario podrá modificar parámetros de los nodos.	Baja
RF-027	Configuración de ajustes	El usuario podrá modificar los ajustes de conexión, del sistema de reconocimiento de marcas visuales, lenguaje, etc.	Alta

3.4.2. Requisitos no funcionales

ID	Categoría	Descripción	Prioridad
RNF-001	Usabilidad	Deben evitarse las interacciones innecesarias del usuario priorizando la configuración automática cuando sea posible para reducir carga cognitiva.	Alta
RNF-002	Escalabilidad	Los módulos deben desarrollarse teniendo en cuenta la posibilidad de su ampliación y la inclusión futura de sistemas de <i>plugins</i> .	Alta

3.4.3. Casos de uso

CU-01: Configuración conexión ROS

Actor Principal: Usuario

Objetivo: Establecer exitosamente la conexión con ROS.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- Conocer la dirección IP del servidor.
- Conocer el puerto en el que se expone el servicio.

Flujo Principal:

1. El usuario accede a la configuración.
2. El usuario introduce la dirección IP.
3. El usuario introduce el puerto.
4. El usuario modifica otros parámetros como el tiempo de vida de conexión, la conexión en el inicio, etc.
5. El usuario guarda los cambios.
6. El sistema almacena la información y restablece la conexión con los nuevos parámetros.
7. Se visualizan los *frames* presentes en el grafo ROS.

Flujos Alternativos:

- Dirección IP o puerto no válido: El sistema no muestra los *frames*.

CU-02: Configuración idioma

Actor Principal: Usuario

Objetivo: Establecer exitosamente el idioma.

Precondiciones: -

Flujo Principal:

1. El usuario accede a la configuración.
2. El usuario selecciona el idioma deseado.
3. El usuario guarda los cambios.
4. El sistema modifica los textos de la aplicación.

Flujos Alternativos: -

CU-03: Asignación de *frame* a *AprilTag*

Actor Principal: Usuario

Objetivo: Asociar exitosamente un *AprilTag* a un *frame*.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.
- El *AprilTag* debe no haber sido asociado anteriormente o liberado de su asignación.

Flujo Principal:

1. El usuario enfoca la camara del dispositivo hacia un *AprilTag*.
2. El sistema detecta el *AprilTag* y determina si ha sido asignado.
3. Si no ha sido asignado (o se ha liberado), se solicita al usuario que asocie la marca visual a un *frame* concreto.
4. El usuario selecciona el *frame* y guarda los cambios.
5. El sistema almacena la asignación.

Flujos Alternativos:

- Si la marca visual ya está asignada no se solicitará al usuario la validación.

CU-04: Configuración nube de puntos

Actor Principal: Usuario

Objetivo: Establecer exitosamente la configuración de los nube de puntos para los *frames*.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede a la configuración.
2. El usuario selecciona un *frame*.

3. El usuario selecciona el *topic* al que corresponde la nube de puntos.
4. El sistema almacena la información.
5. Se visualiza la nube de puntos en los *frames* correspondientes.

Flujos Alternativos: -

CU-05: Configuración visualización ruta planificada

Actor Principal: Usuario

Objetivo: Establecer exitosamente la visualización de la ruta planificada.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutándose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario enfoca la cámara del dispositivo hacia un *AprilTag*.
2. El sistema detecta el *AprilTag* y determina si ha sido asignado.
3. Si no ha sido asignado (o se ha liberado), se solicita al usuario que asocie la marca visual a un *frame* concreto.
4. El usuario selecciona el *topic* de la visualización de ruta planificada y guarda los cambios.
5. El sistema almacena la asignación.

Flujos Alternativos:

- Si la marca visual ya está asignada no se solicitará al usuario la validación.

CU-06: Visualizar/Ocultar *frames*

Actor Principal: Usuario

Objetivo: Realizar exitosamente la ocultación/visualización de un *frame*.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutándose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede a la configuración.
2. El usuario selecciona el *frame* objetivo.
3. El usuario activa/desactiva el *frame*.
4. El sistema visualiza/oculta el *frame* seleccionado.

Flujos Alternativos: -**CU-07: Teleoperación**

Actor Principal: Usuario

Objetivo: Realizar exitosamente la operación telemática de un robot.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede a la teleoperación.
2. El usuario selecciona el *topic* objetivo del movimiento.
3. El usuario acciona los controles virtuales.
4. El sistema envía el mensaje al *topic* seleccionado.
5. El robot se mueve correspondiendo a las ordenes realizadas.

Flujos Alternativos: -**CU-08: Planificar ruta**

Actor Principal: Usuario

Objetivo: Realizar exitosamente la planificación de la ruta de un robot.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede a la planificación de ruta.
2. El usuario selecciona el *topic* objetivo de la planificación.
3. El usuario añade/elimina puntos objetivos para el robot.
4. El usuario ejecuta la ruta.
5. El sistema envía punto a punto la ruta al *topic* objetivo.

Flujos Alternativos: -**CU-09: Medición entre *frames***

Actor Principal: Usuario

Objetivo: Realizar exitosamente la medición de la distancia entre *frames*.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutándose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede a la medición.
2. El usuario selecciona un *frame* de los presentes.
3. El usuario selecciona otro *frame*.
4. El sistema calcula la distancia, la rotación, etc.

Flujos Alternativos: -**CU-10: Ajuste de posición de *frames* asignados a marcas visuales**

Actor Principal: Usuario

Objetivo: Realizar exitosamente el ajuste de la posición de *frames* en las marcas visuales.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutándose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede al ajuste de *frames*.
2. El usuario ajusta un *frame* raíz.
3. El sistema almacena el ajuste y lo aplica constantemente.

Flujos Alternativos: -**CU-11: Ajuste de escala de *frames* asignados a marcas visuales**

Actor Principal: Usuario

Objetivo: Realizar exitosamente el ajuste de la escala de un *frame*.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede al ajuste de *frames*.
2. El usuario selecciona un *frame*.
3. El usuario aumenta o disminuye la escala de un *frame*.
4. El sistema almacena el ajuste y lo aplica constantemente.

Flujos Alternativos: -**CU-12: Suscripción a *topics***

Actor Principal: Usuario

Objetivo: Realizar exitosamente la suscripción a un *topic*.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede a la suscripción.

2. El usuario selecciona un *topic*.
3. El usuario verifica la selección.
4. El sistema se suscribe al *topic* y muestra los mensajes al usuario.

Flujos Alternativos: -

CU-13: Publicación en *topics*

Actor Principal: Usuario

Objetivo: Realizar exitosamente la publicación de un mensaje en un *topic*.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede a la publicación.
2. El usuario selecciona un *topic*.
3. El usuario selecciona un tipo de mensaje.
4. El usuario introduce el mensaje.
5. El usuario introduce selecciona la frecuencia de publicación.
6. El usuario verifica la selección.
7. El sistema se publica el mensaje al *topic* seleccionado.

Flujos Alternativos: -

CU-14: Modificación de parámetros en nodos

Actor Principal: Usuario

Objetivo: Realizar exitosamente la modificación de un parámetro en un nodo.

Precondiciones:

- El servidor auxiliar de la aplicación debe estar ejecutandose dentro de la misma red o en otra.
- La conexión debe haberse establecido de forma exitosa.

Flujo Principal:

1. El usuario accede a la modificación de parámetros.
2. El usuario selecciona un parámetro.
3. El usuario introduce el nuevo del valor del parámetro.
4. El sistema envía el valor del parámetro al nodo.

Flujos Alternativos: -

3.5. Diseño del sistema

En este apartado se muestran los diagramas de los módulos principales de la aplicación, los cuales se describen en el apartado de implementación del sistema.

En la Figura 3.1 se muestra el diagrama del módulo de transformaciones de ROS, el cual se encarga de reconstruir la jerarquía de ROS en Unity y mostrarla al usuario mediante diferentes funcionalidades (visualización de velocidades, saltos, *trail*, etc.).

La Figura 3.2 muestra el diagrama del módulo de nubes de puntos, el cual se encarga de mostrar en el mundo virtual las nubes de puntos emitidas por los sensores de detección de profundidad como *LiDAR*.

En la Figura 3.3 se muestra el diagrama del módulo de control espacial. Este se encarga de permitir la planificación de rutas y visualizarlas en el mundo virtual.

El módulo de suscripción a temas de ROS se muestra en la Figura 3.4. Este se encarga de establecer la comunicación con los nodos de ROS y recibir los mensajes publicados en los *topics* a los que se suscribe, para ello, se emplea la clase base *ROSSubscriber* que permite la suscripción a cualquier *topic* de ROS.

De forma similar al módulo de suscripción a temas, el módulo de publicación de temas de ROS se muestra en la Figura 3.5. Este es el encargado de gestionar la comunicación con los servicios publicados en los nodos, para ello, se emplea la clase base *ROSService* que permite la invocación de cualquier servicio de ROS.

3.6. Implementación del sistema

La aplicación está dividida en diferentes módulos con diversas interdependencias. Entre ellos destacan principalmente el de detección de *AprilTags*, es decir, el sistema de detección de puntos de referencia, la visualización de las transformaciones – y sus consecuentes datos como velocidad angular y lineal, posición, rotación, *jitter*, etc. – y el control y visualización espacial de los robots.

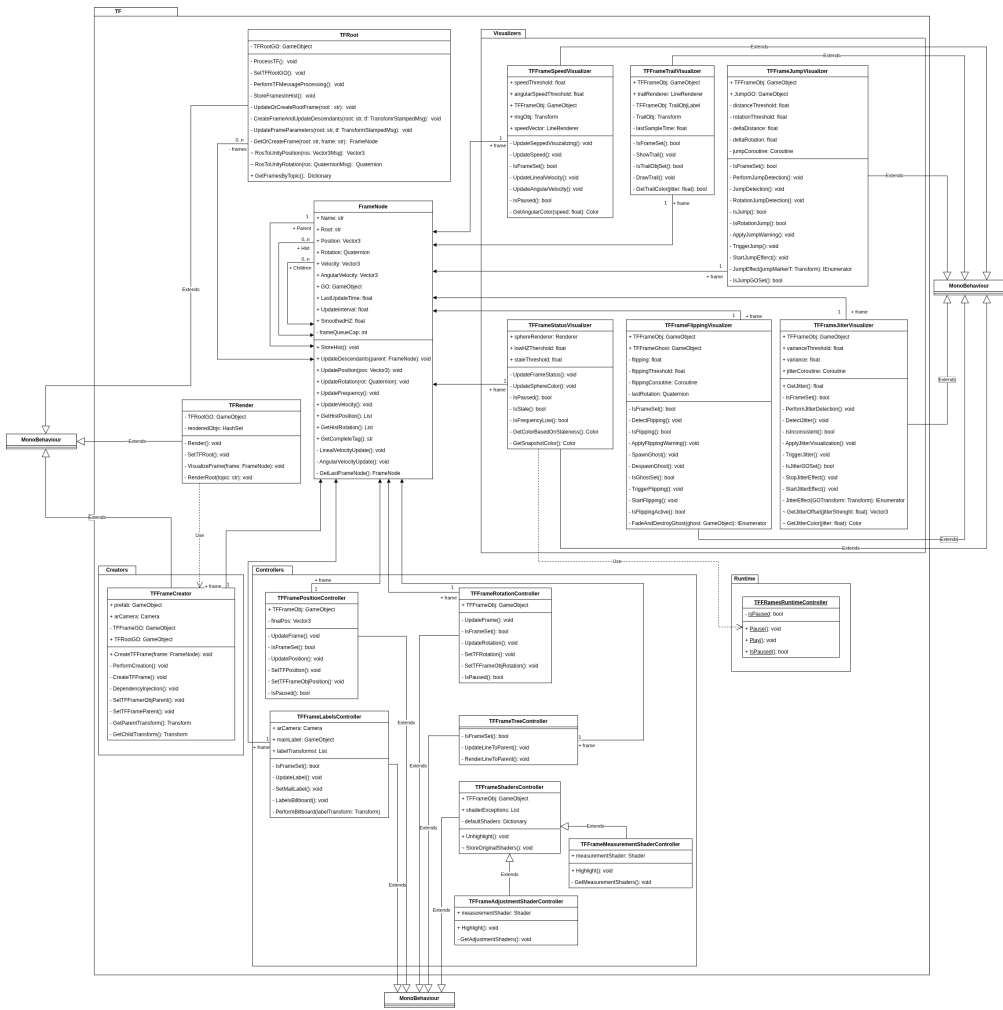


Figura 3.1: Módulo TF

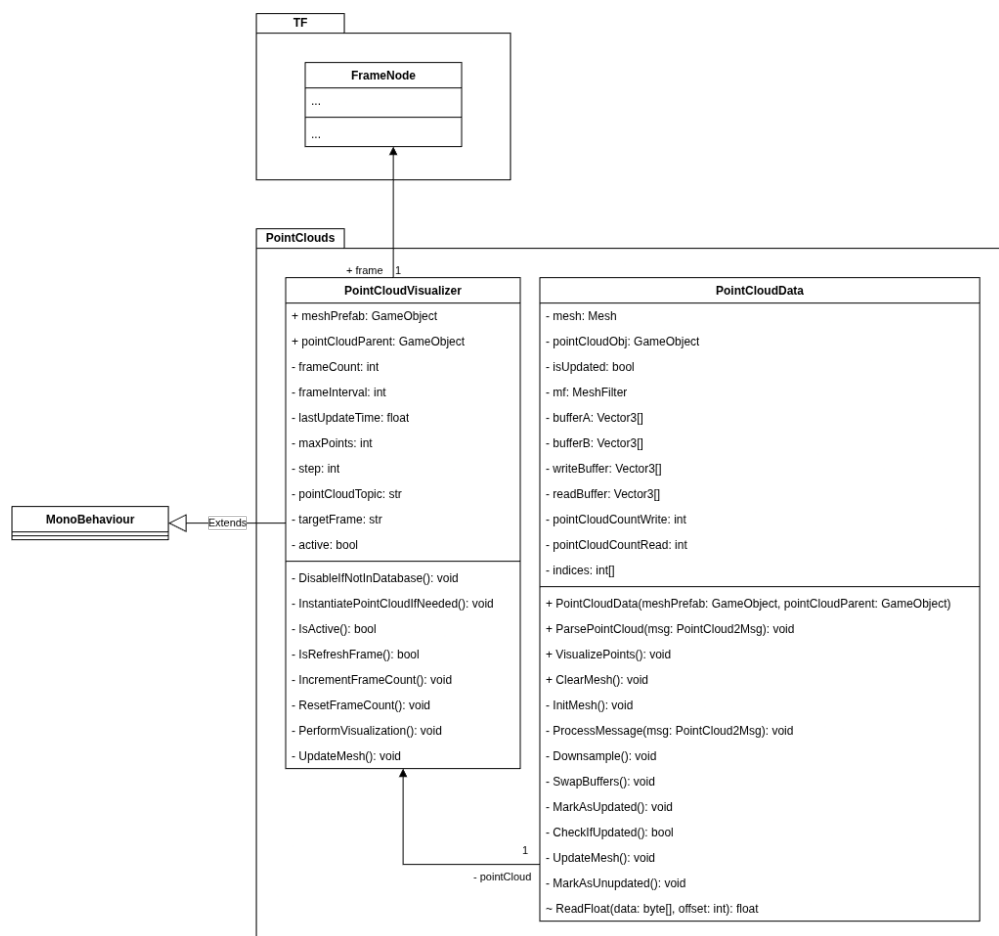


Figura 3.2: Módulo PointClouds

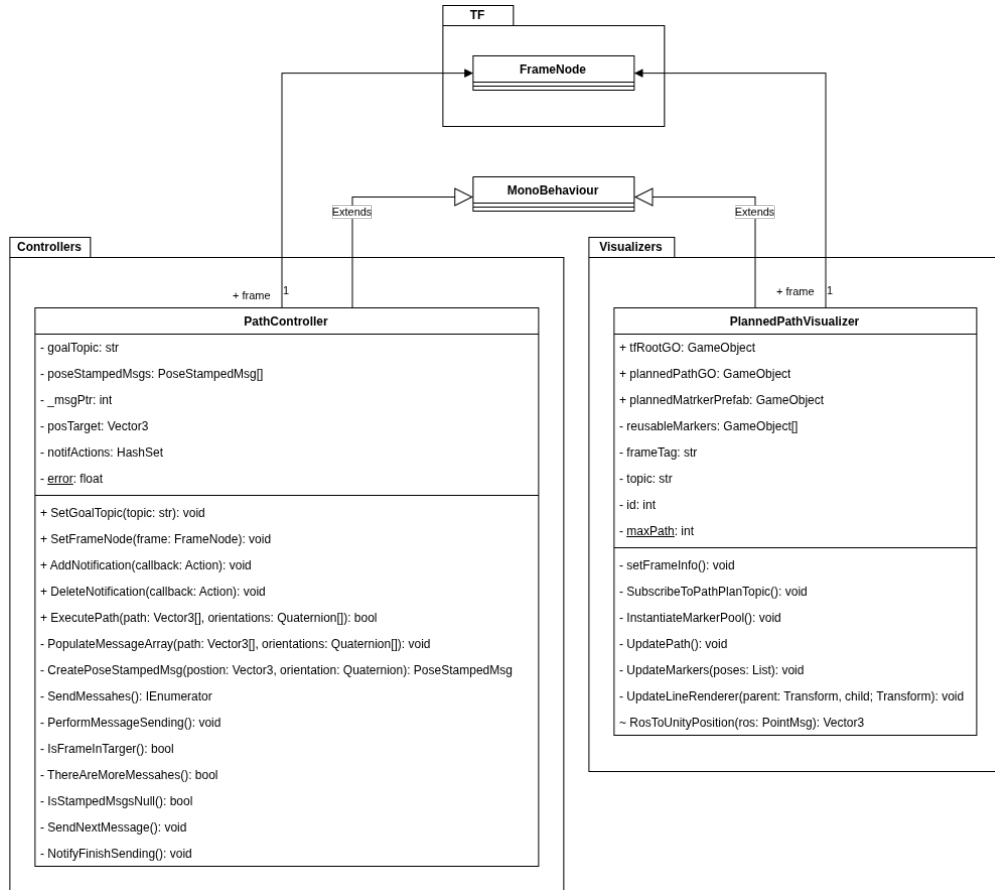


Figura 3.3: Módulo SpatialControl

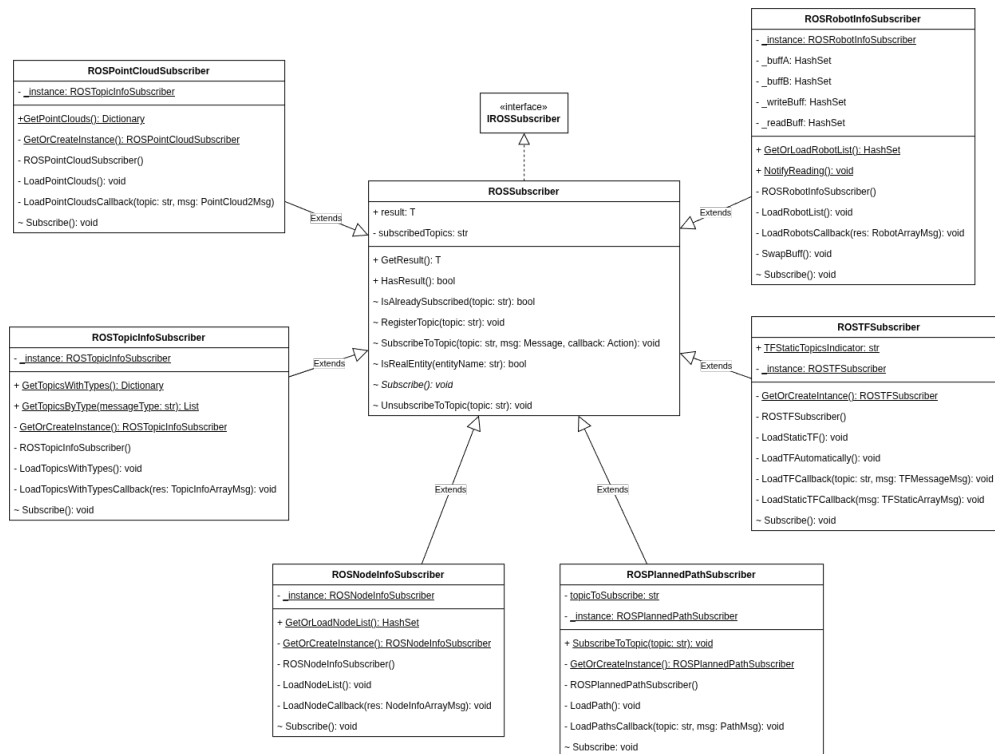


Figura 3.4: Módulo ROSSubscriber

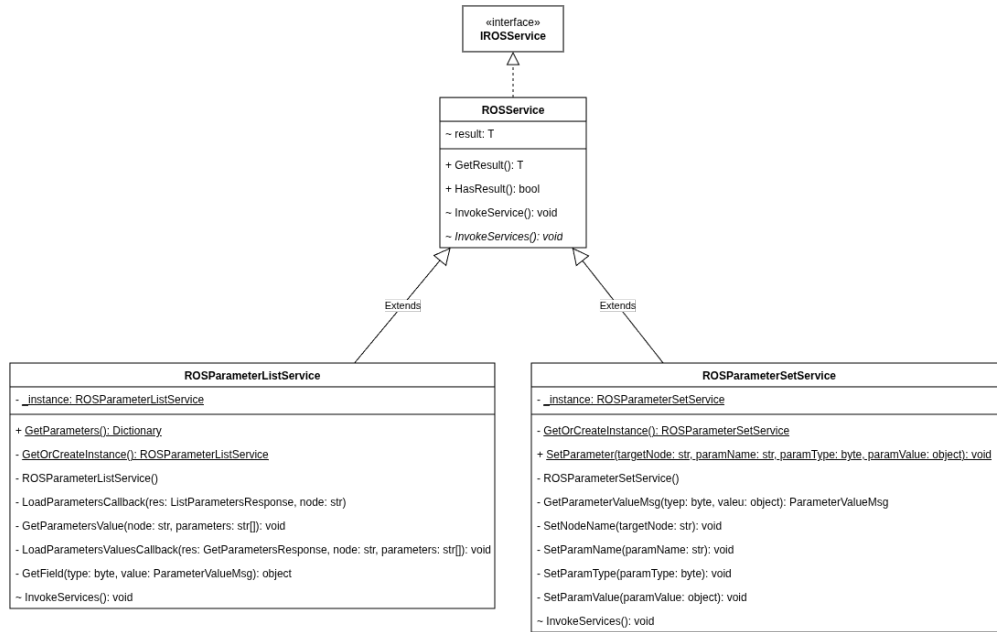


Figura 3.5: Módulo ROService

En cuanto al primer módulo, se trata de la librería **AprilRobotics/apriltag** desarrollada en C compilada como dependencia externa para ser empleada como *plugin* en Unity. Cada *frame* de la cámara de la aplicación se transforma a escala de grises, se invierte para acomodarse a los requisitos de la aplicación y se envía al *plugin* donde se identifican las etiquetas presentes en la imagen. Esta dependencia devuelve la estructura de datos con la información necesaria de las etiquetas – en este caso: el identificador y la posición con respecto a la imagen –, de forma que si una etiqueta es nueva en el sistema (no ha sido almacenada en la base de datos de etiquetas), se interpela al usuario para que realice la asociación con el *frame* de las transformaciones de ROS y el *topic* empleado en la visualización del camino proyectado correspondiente. Este módulo puede considerarse, junto al de la visualización de las transformaciones, como el núcleo principal de la aplicación, puesto que es el encargado de hacer converger el mundo virtual con el físico.

La visualización de las transformaciones de ROS se lleva a cabo en el módulo **TF**. Este se encarga reconstruir la jerarquía de ROS en Unity procesando los mensajes que los nodos publican en los *topics* de transformación – aquellos que contienen mensajes del tipo *TFMessage* – y creando los objetos de juego (*GameObjects*) que permitirán visualizar diferentes datos como las orientaciones de un brazo robótico, la velocidad lineal de un robot móvil o su velocidad angular y problemas característicos en la robótica como el *jitter*, los saltos, el *trail*, *flipping*, etc. Es el módulo que permite conocer el estado del sistema en todo momento, lo que da el sentido al resto de funcionalidades de la aplicación.

La visualización espacial puede considerarse una complementación al módulo anterior, puesto que es el encargado de situar en el mundo mediante la realidad aumentada el estado de los sensores de detección de profundidad como *LiDAR*. Este se trata de un módulo de

inicialización activa, es decir, en el caso de que el robot cuente con un sensor que emita mensajes *PointCloud2*, el usuario debe asociar al *frame* correspondiente el *topic* donde se publiquen dichos mensajes. Se estudió la manera de realizar este emparejamiento de forma automática, pero debido a que el usuario final puede no seguir una convención concreta de nombres en los *topics*, es preferible que sea este quien realice la confirmación de la información; de otra manera, la información que se muestre puede ser errónea y eventualmente propiciar accidentes.

El control espacial permite que robots móviles puedan ser operados empleando la aplicación mediante control directo (teleoperación) o planificación de ruta (*path planning*). La operación telemática esta compuesta por un sistema multimodal que permite, mediante el accionamiento de un *joystick* y botones virtuales, el control total de un robot concreto. La planificación de ruta, a su vez, permite situar de forma virtual marcas en las superficies interactuables en realidad aumentada – aquellas captadas por las cámaras y trasformadas en planos dentro del mundo digital – que posteriormente son transformadas en localizaciones del mundo real a las que los robots móviles acudirán cumpliendo la ruta programada. Puesto que ejecutar la acción del movimiento cuando el usuario posiciona una marca de referencia no es seguro ni práctico ni sigue la filosofía de la planificación, el usuario será quien decida cuando ejecutar cada plan en cada robot.

Además de los mencionados anteriormente, existen submódulos que proporcionan funcionalidades que los usuarios pueden considerar como de gran importancia para la consecución de sus objetivos con la aplicación. Entre ellos destacan los módulos de publicación y observación de mensajes en *topics*. Con estas funcionalidades pueden realizarse acciones en robots que no tienen actualmente una función específica desarrollada en la aplicación o suscribirse a cadenas de mensajes en un robot para observar qué es lo que le ocurre en caso de problemas no descubiertos.

3.7. Validación y pruebas

3.7.1. Pruebas del CU-01: Configuración conexión ROS

ID	Datos de entrada	Resultado esperado
CP-01	IP y puerto válidos	Conexión establecida
CP-02	IP incorrecta	Mensaje de error
CP-03	Puerto incorrecto	Mensaje de error
CP-04	Campos vacíos	Validación de campos obligatorios

Tabla 3.3: Casos de prueba de la configuración de conexión con ROS

3.7.2. Pruebas del CU-02: Configuración idioma

ID	Datos de entrada	Resultado esperado
CP-05	Castellano seleccionado	Cambio textos aplicación a español
CP-06	Inglés seleccionado	Cambio textos aplicación a inglés

Tabla 3.4: Casos de prueba de la configuración de idioma

3.7.3. Pruebas del CU-03: Asignación de *frame* a *AprilTag*

ID	Datos de entrada	Resultado esperado
CP-07	<i>Frame</i> seleccionado	Asociación <i>AprilTag-frame</i> establecida

Tabla 3.5: Casos de prueba de la asignación de *frame* a *AprilTag*

3.7.4. Pruebas del CU-04: Configuración de nube de puntos

ID	Datos de entrada	Resultado esperado
CP-08	Selección <i>topic</i> para un <i>frame</i>	Asociación <i>frame-topic</i> de nube de puntos establecida (visualización de la nube de puntos en el <i>frame</i>)

Tabla 3.6: Casos de prueba de la configuración de nube de puntos

3.7.5. Pruebas del CU-05: Configuración visualización ruta planificada

ID	Datos de entrada	Resultado esperado
----	------------------	--------------------

CP-09	Selección <i>topic</i> para el <i>AprilTag</i>	Asociación <i>AprilTag-topic</i> de ruta planificada establecida (visualización de la ruta en caso de existir)
-------	--	--

Tabla 3.7: Casos de prueba de configuración de visualización de ruta planificada

3.7.6. Pruebas del CU-06: Visualizar/Ocultar *frames*

ID	Datos de entrada	Resultado esperado
CP-10	Selección ocultación <i>frame</i>	<i>Frame</i> deja de visualizarse al usuario
CP-11	Selección visualización <i>frame</i>	<i>Frame</i> se visualiza al usuario

Tabla 3.8: Casos de prueba de visualización/ocultación de *frames*

3.7.7. Pruebas del CU-07: Teleoperación

ID	Datos de entrada	Resultado esperado
CP-12	Mensaje de avance	Robot se mueve a velocidad lineal establecida hacia adelante
CP-13	Mensaje de retroceso	Robot se mueve a velocidad lineal establecida hacia atrás
CP-14	Mensaje de giro hacia izquierda	Robot se gira a velocidad angular establecida hacia izquierda
CP-15	Mensaje de giro hacia derecha	Robot se gira a velocidad angular establecida hacia derecha

CP-16	Mensaje de subida	Robot sube el elemento correspondiente a velocidad establecida
CP-17	Mensaje de bajada	Robot baja el elemento correspondiente a velocidad establecida

Tabla 3.9: Casos de prueba de teleoperación

3.7.8. Pruebas del CU-08: Planificar ruta

ID	Datos de entrada	Resultado esperado
CP-18	Posición de marca de referencia añadida	La marca se visualiza encadenada a la anterior (o al robot en caso de ser la primera) en la posición establecida
CP-19	Accionamiento de eliminación de marca de referencia	La ultima marca se deja de visualizar
CP-20	Mensaje de la ruta planificada	Robot se desplaza consecutivamente a todos los puntos de la ruta

Tabla 3.10: Casos de prueba de planificación de ruta

3.7.9. Pruebas del CU-09: Medición entre *frames*

ID	Datos de entrada	Resultado esperado
CP-21	Selección primer <i>frame</i>	<i>Frame</i> se visualiza en otro color y textura
CP-22	Selección segundo <i>frame</i>	<i>Frame</i> se visualiza en otro color y textura y se muestra la distancia, giro, etc. entre ellos

 Tabla 3.11: Casos de prueba de medición entre *frames*

3.7.10. Pruebas del CU-10: Ajuste de posición de *frames* asignados a marcas visuales

ID	Datos de entrada	Resultado esperado
CP-23	Selección <i>frame</i>	<i>Frame</i> se visualiza en otro color y textura
CP-24	Mensaje de desplazamiento a izquierda	<i>Frame</i> junto con hijos se desplazan hacia la izquierda
CP-25	Mensaje de desplazamiento a derecha	<i>Frame</i> junto con hijos se desplazan hacia la derecha
CP-26	Mensaje de desplazamiento hacia delante	<i>Frame</i> junto con hijos se desplazan hacia delante
CP-27	Mensaje de desplazamiento hacia atrás	<i>Frame</i> junto con hijos se desplazan hacia atrás
CP-28	Mensaje de desplazamiento hacia arriba	<i>Frame</i> junto con hijos se desplazan hacia arriba
CP-29	Mensaje de desplazamiento hacia abajo	<i>Frame</i> junto con hijos se desplazan hacia abajo

 Tabla 3.12: Casos de prueba del ajuste de la posición de *frames* asignados a marcas visuales

3.7.11. Pruebas del CU-11: Ajuste de escala de *frames* asignados a marcas visuales

ID	Datos de entrada	Resultado esperado
CP-30	Selección <i>frame</i>	<i>Frame</i> se visualiza en otro color y textura

CP-31	Aumento de escala	<i>Frame</i> se visualiza más grande
CP-32	Disminución de escala	<i>Frame</i> se visualiza más pequeño

Tabla 3.13: Casos de prueba del ajuste de escala de *frames* asignados a marcas visuales

3.7.12. Pruebas del CU-12: Suscripción a *topics*

ID	Datos de entrada	Resultado esperado
CP-33	Selección <i>topic</i>	Se visualizan los mensajes publicados por los nodos en el <i>topic</i>

Tabla 3.14: Casos de prueba de la suscripción a *topics*

3.7.13. Pruebas del CU-13: Publicación en *topics*

ID	Datos de entrada	Resultado esperado
CP-34	Mensaje mal formulado	Se muestra mensaje de error
CP-35	Frecuencia errónea	El sistema corrige la entrada de usuario estableciendo un valor válido

Tabla 3.15: Casos de prueba de la publicación en *topics*

3.7.14. Pruebas del CU-14: Modificación de parámetros en nodos

ID	Datos de entrada	Resultado esperado
CP-36	Valor erróneo de parámetro	Se muestra mensaje de error

Tabla 3.16: Casos de prueba de la modificación de parámetros

3.8. Consideraciones éticas y legales

Todas las etapas de desarrollo del proyecto han sido enfocadas considerando a los usuarios como el núcleo central de la usabilidad dentro de la aplicación, teniendo en cuenta de forma primordial la reducción de la carga cognitiva necesaria para la toma de decisiones, a la par que se intenta salvaguardar sus intereses, integridad física y psicológica. Así pues, se prima la simplificación en los procesos repetitivos y tediosos como la navegación excesiva por menús, optando por elementos interactivables sencillos y visuales; sin embargo, se evita realizar suposiciones ante situaciones que puedan conllevar un riesgo hacia la integridad del usuario, por ello la aplicación cuenta con diferentes configuraciones en las que se solicita la verificación de los datos.

En cuanto al licenciamiento *software*, se ha optado por incluir dependencias de las que se pueda comprobar su trazabilidad sin impedimento, de manera que cualquier desarrollador o persona con conocimientos de programación pueda verificar la integridad e intencionalidad de los algoritmos utilizados. Por ello, durante el desarrollo del proyecto se emplearán librerías *opensource* con licencias *MIT*, *BSD* y *Apache2* entre otras.

El resultado final, de igual forma, deberá poder ser trazado libremente y emplearse en contextos de forma que se salvaguarden las condiciones de uso de sus dependencias. Contará, de esta manera, con una licencia no más restrictiva que las dependencias que lo componen.

Cabe destacar, por otro lado, que algunos iconos de la aplicación han sido generados empleando inteligencia artificial generativa al ser imágenes sencillas y reproducibles fácilmente por un ser humano.

Capítulo 4

Resultados

4.1. Software desarrollado

En este apartado se mostrará de manera fotográfica las funcionalidades desarrolladas en la aplicación.

En la Figura 4.1 puede observarse cómo se visualizan los *frames* dentro de la aplicación y su relación jerárquica. El primero se dibuja como un objeto en el que se muestran los diferentes ejes de coordenadas (verde para eje Z, rojo para eje X y azul para eje Y) y una esfera central en la que se indica el estado de la comunicación. La relación jerárquica se visualiza mediante una línea roja desde el elemento hijo al padre. El selector superior indica el *frame* objetivo de ciertas operaciones en diferentes funcionalidades y será un elemento constante en toda la aplicación.

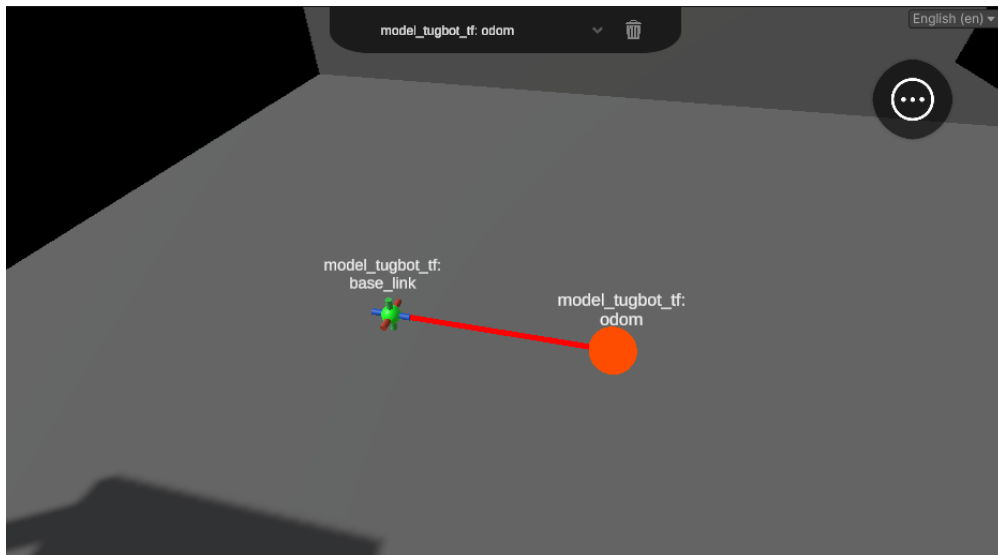


Figura 4.1: Visualización *transforms*

En la Figura 4.2 se puede observar cómo se muestra en la aplicación la velocidad lineal. Esta variable es visualizada empleando un vector de color azul en el que la longitud indica la

dirección del desplazamiento, de manera que cuanto mayor sea el vector, a mayor velocidad se desplazará el *frame*. Además, en la parte posterior del movimiento se puede observar otro vector que indica la magnitud de la velocidad, de forma que mediante un código de colores puede determinarse su adecuación. Así pues, el verde indica que la velocidad lineal es menor a 0.5m/s, el amarillo que está entre 0,5m/s y 1m/s y rojo que está por encima del último umbral mencionado.

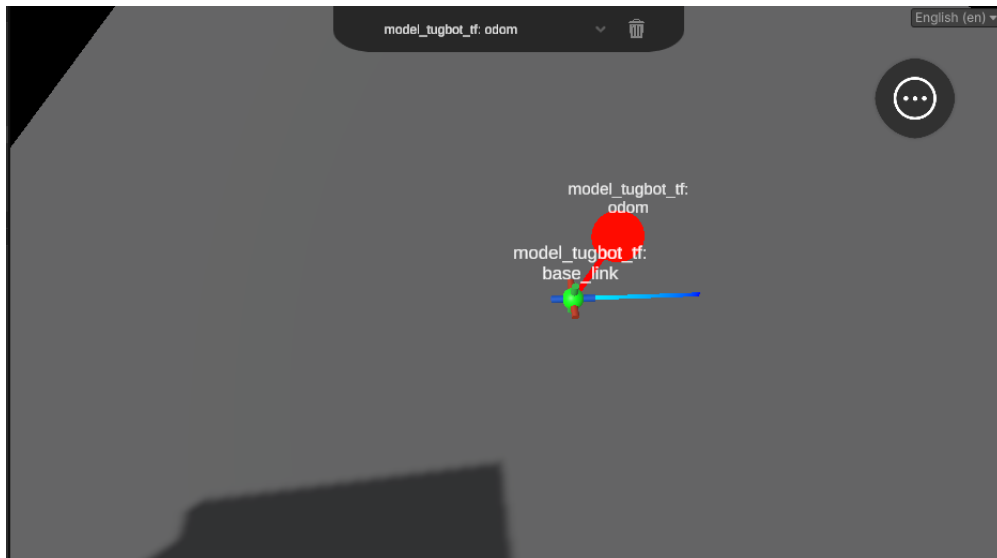


Figura 4.2: Visualización velocidad lineal

En la Figura 4.3 se puede observar cómo se muestra en la aplicación la velocidad angular. Esta variable es visualizada mediante un elemento toroidal que indica el eje en el que se está realizando el giro; el color, a su vez, indica que la velocidad se encuentra dentro de ciertos márgenes (verde para velocidades menores a 0,5 rad/s, amarillo para velocidades entre 0,5 rad/s y 1 rad/s).

Las Figuras 4.4 y 4.5 muestra la funcionalidad de visualización y ocultación de *frames*. La aplicación proporcionará la lista de *frames* y el usuario seleccionará aquellos que desee ocultar/mostrar

La Figura 4.6 muestra la funcionalidad de teleoperación de robots. El usuario deberá seleccionar el *topic* objetivo de la comunicación y mediante el accionamiento del *joystick* virtual o los botones enviará mensajes de movimiento. La aplicación permite, además, modificar parámetros de la velocidad lineal, vertical y angular.

Similar a la funcionalidad anterior es la planificación de rutas. La Figura 4.7 muestra su interfaz. El usuario, de igual manera, seleccionará el *topic* objetivo de la comunicación y el *frame* objetivo empleando los selectores inferiores y superiores respectivamente. Posteriormente, seleccionará uno o varios puntos en el mundo en el que se situarán marcadores azules unidos por una línea indicando el recorrido a realizar. Esta ruta no será ejecutada hasta que el usuario no envíe el mensaje mediante el botón con el avión de papel como icono. El botón

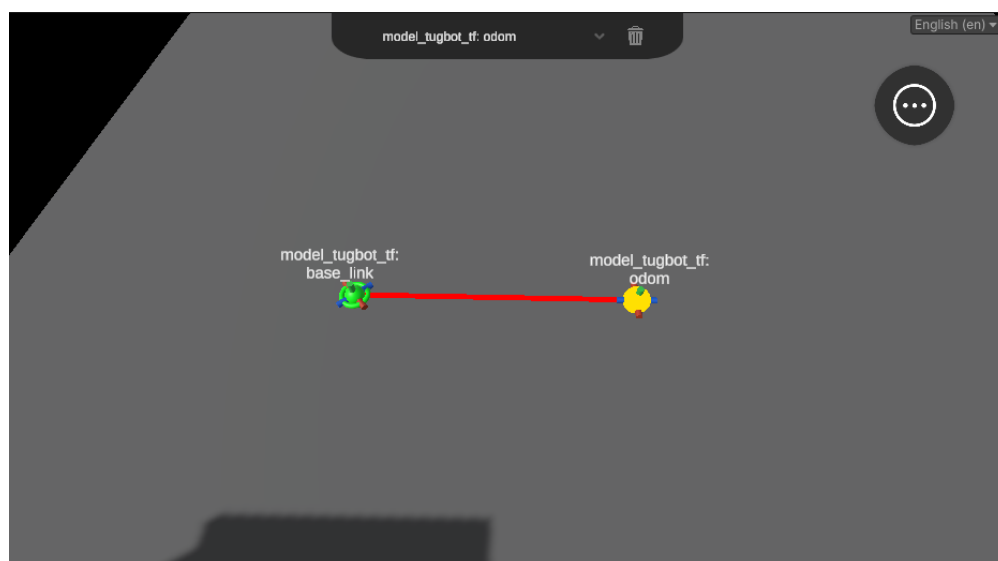


Figura 4.3: Visualización velocidad angular

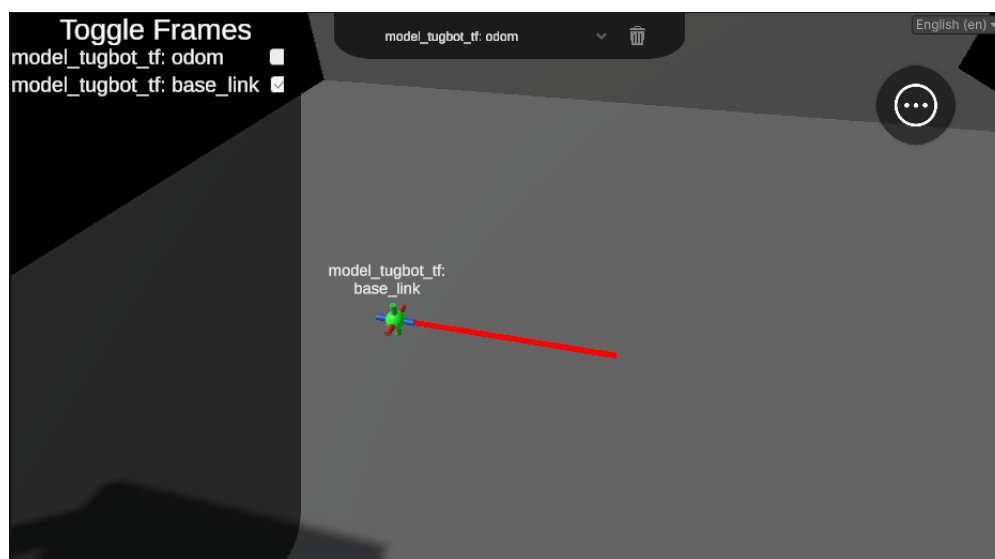


Figura 4.4: Ocultación de frames

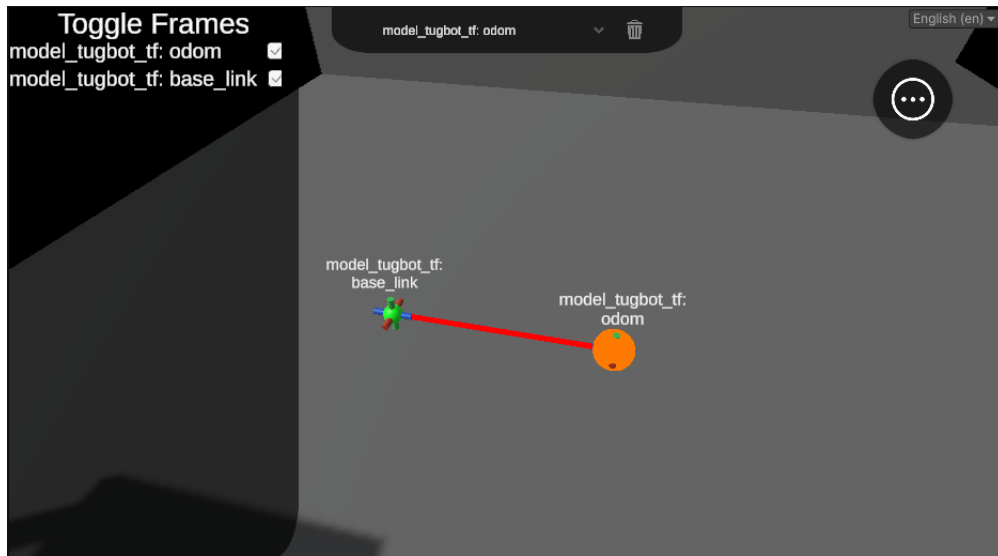


Figura 4.5: Visualización de frames

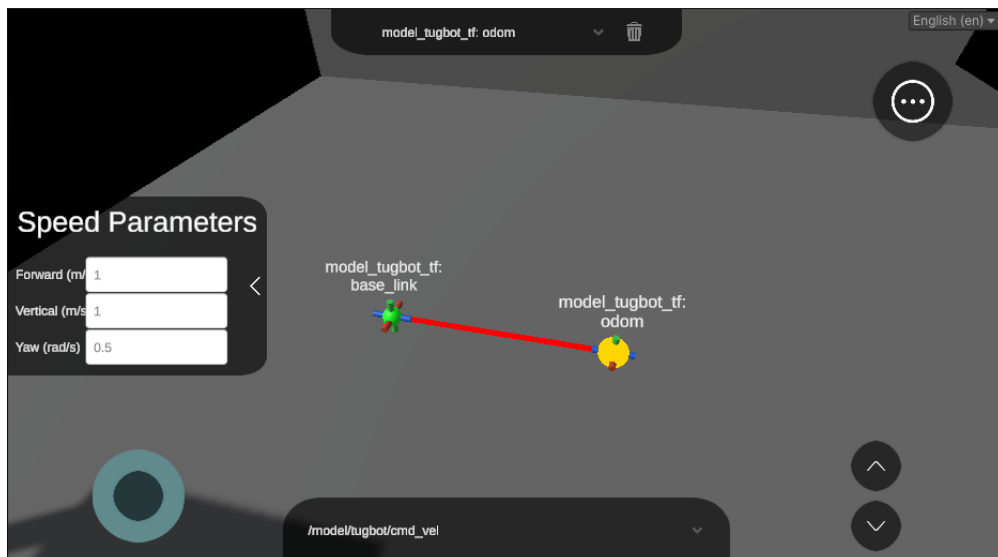


Figura 4.6: Teleoperación

de la izquierda eliminará uno a uno los marcadores, de modo que el usuario puede rehacer la ruta a su conveniencia.

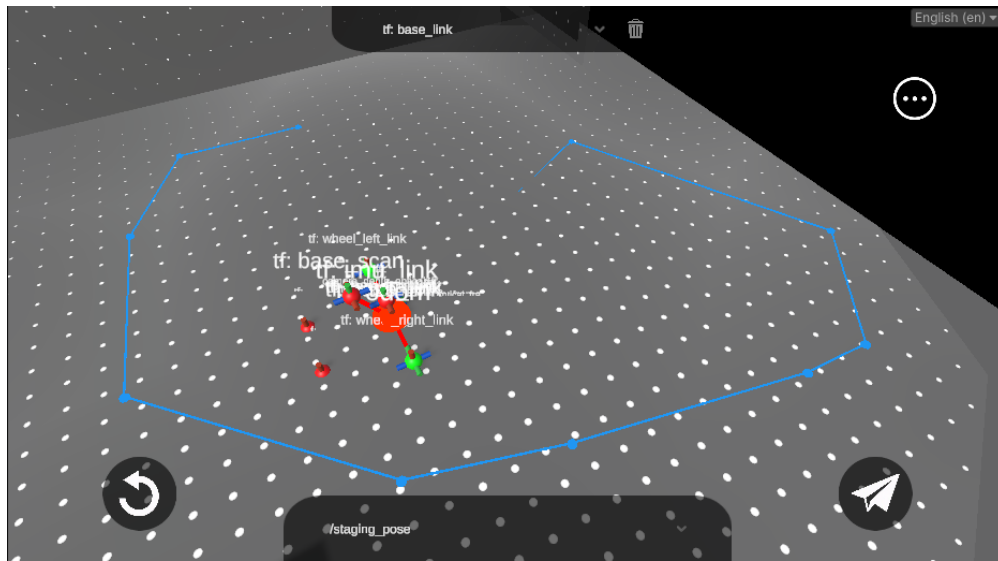


Figura 4.7: Planificación de ruta

La Figura 4.8 muestra la funcionalidad de medidas entre frames. El usuario seleccionará dos *frames*; estos cambiarán de color a azul parpadeante y se mostrará en la ventana izquierda diferentes parámetros como su distancia en metros y su rotación.

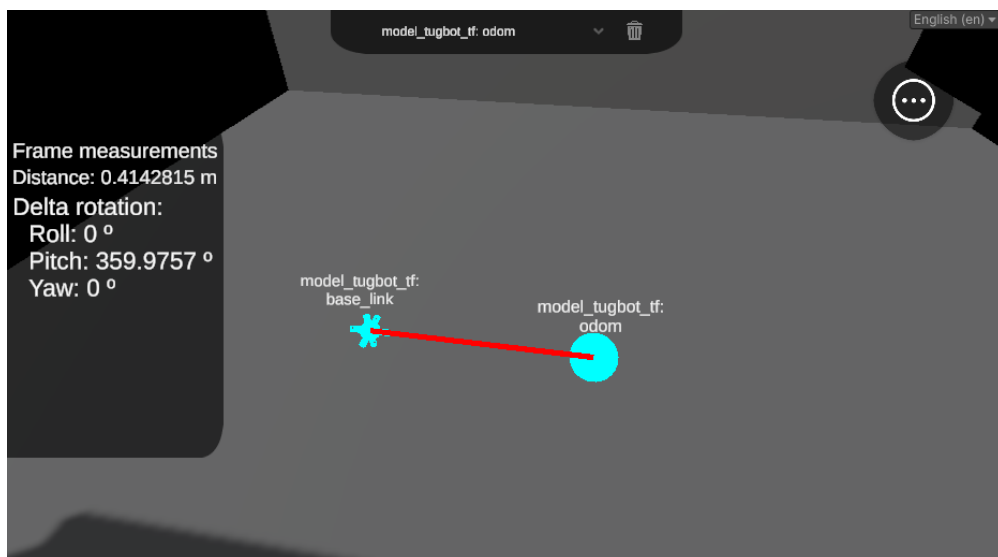


Figura 4.8: Medición de frames

La Figura 4.9 muestra la funcionalidad de escalado de los *frames* en la que el usuario seleccionará un *frame* dejando el dedo unos instantes sobre la pantalla. El objeto cambiará entonces de color a modo transparente parpadeante y será entonces cuando el usuario mediante un pellizco en la pantalla podrá ampliar y reducir la escala del elemento.

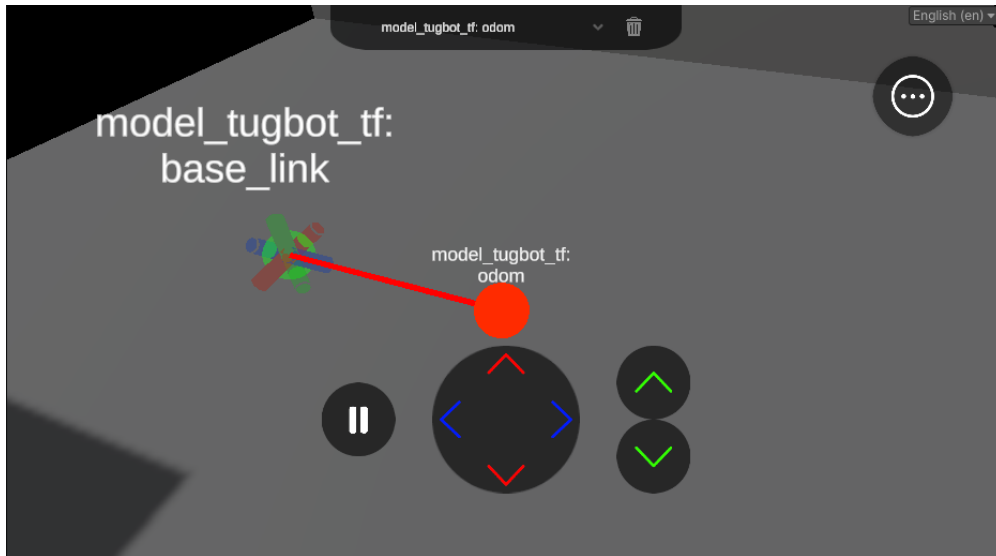


Figura 4.9: Ajuste de escala de frames

Las Figuras 4.10 y 4.11 muestran la funcionalidad de la visualización de nube de puntos. Inicialmente, el usuario debe configurar la asociación entre el *frame* y el *topic* de nube de puntos puesto que se trata de una relación que no puede presuponerse (como se indicó en el apartado de la implementación del sistema). Una vez realizada la asociación, la nube de puntos se visualizará en el *frame* correspondiente.

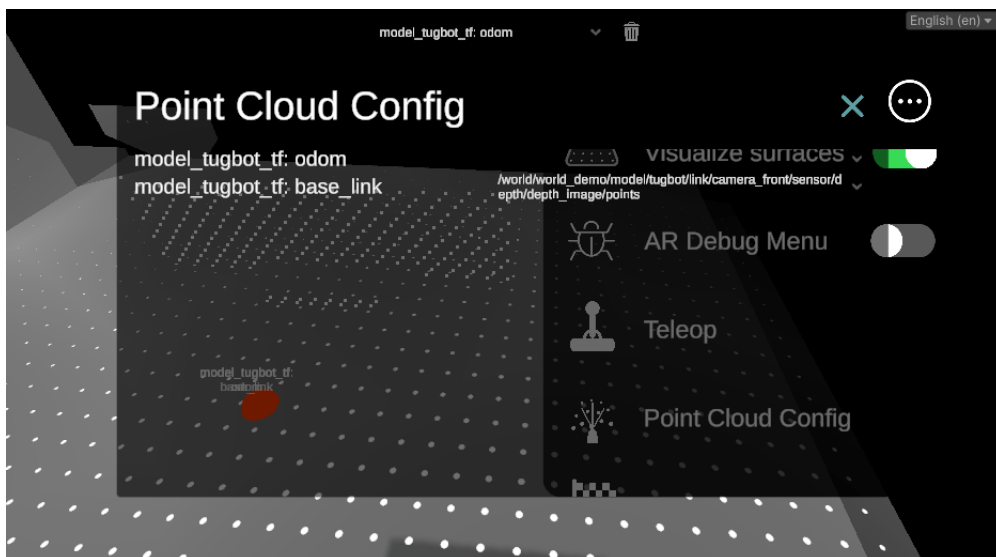


Figura 4.10: Configuración de nube de puntos

La Figura 4.12 muestra la funcionalidad de suscripción a tema para su visualización directa en la aplicación. El usuario seleccionará un *topic* y un tipo de mensaje de los selectores y pulsará en confirmar. Tras esto, los mensajes publicados en ese tema se visualizarán en la parte derecha de la pantalla. Para parar la visualización el usuario puede pulsar en "Parar." "Stop".

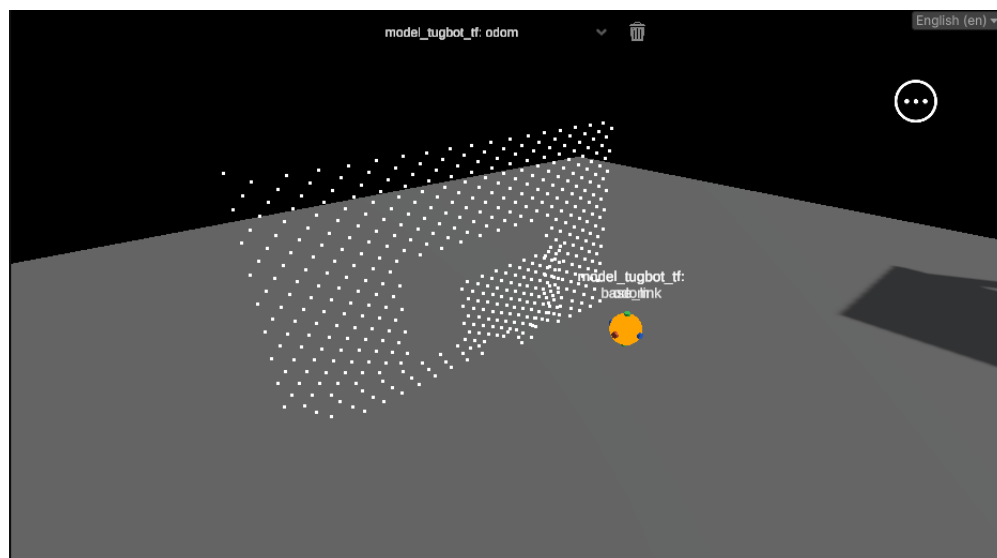
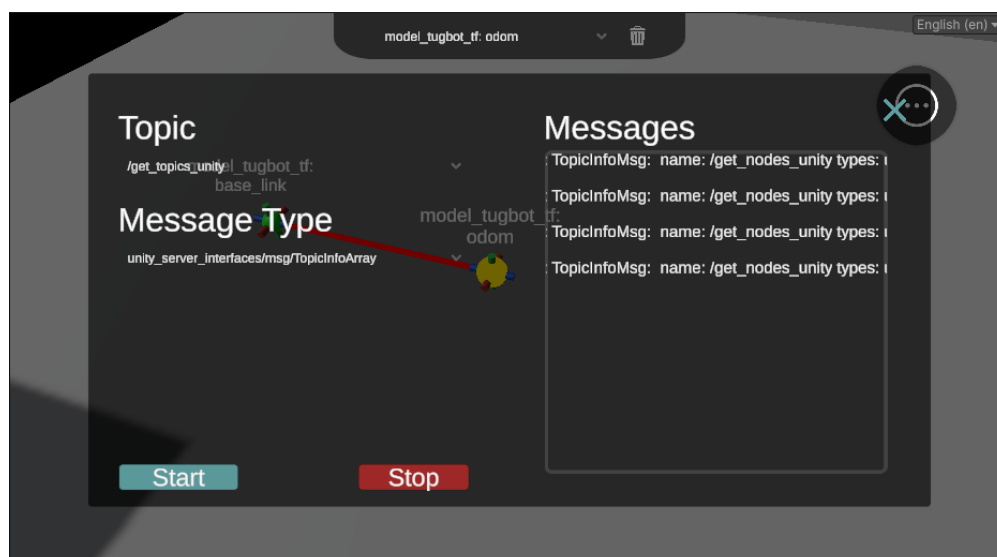


Figura 4.11: Visualización de nube de puntos

Figura 4.12: Suscripción a *topics*

La Figura 4.13 muestra la funcionalidad de publicación en un tema. El usuario seleccionará un *topic* de los disponibles, un tipo de mensaje y cumplimentará el mensaje. Tras seleccionar la frecuencia de publicación en el tema confirmará la publicación y el mensaje será enviado siguiendo los parámetros correspondientes al *topic* correspondiente.

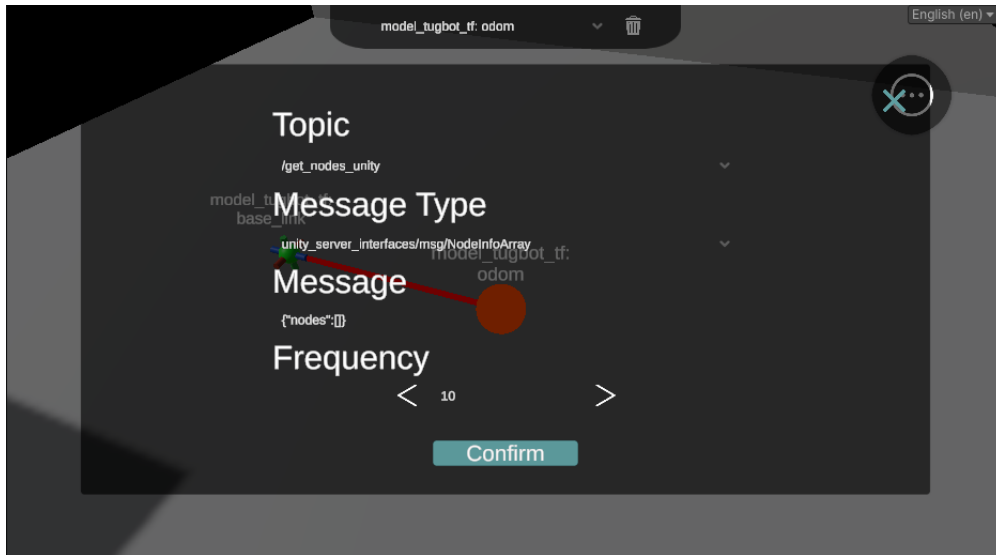


Figura 4.13: Publicación en *topic*

La Figura 4.14 muestra la funcionalidad de configuración de parámetros de los nodos de ROS. El usuario seleccionará el nodo objetivo, el parámetro que quiera modificar e introducirá el nuevo valor siguiendo la guía del valor anterior. Una vez cumplimentado los datos correspondientes, podrá enviar el nuevo valor pulsando "Enviar" o "Save".

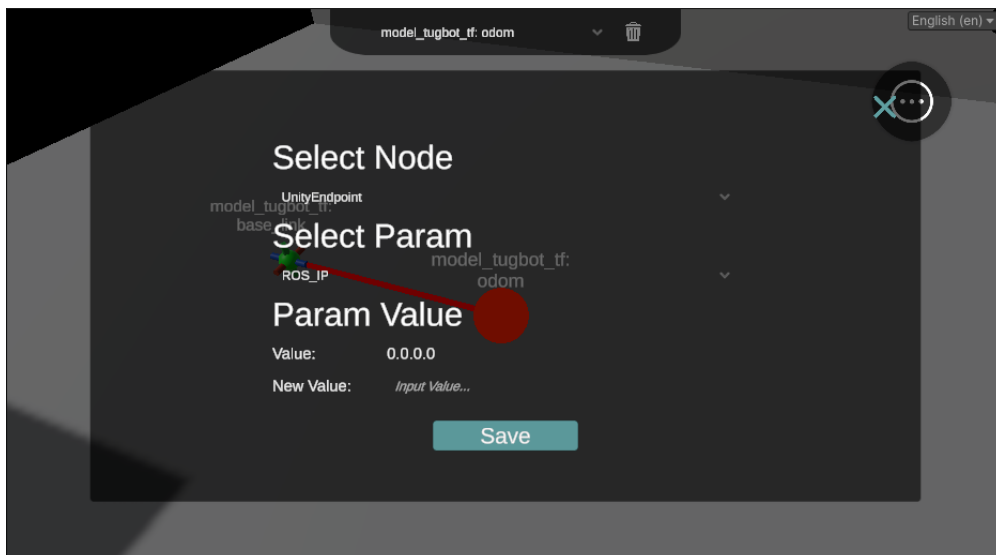


Figura 4.14: Configuración de parámetros

La Figura 4.15 muestra la interfaz de configuración de la aplicación. En esta se podrá

modificar el idioma de la aplicación, el estándar *AprilTag* a emplear en la aplicación y parámetros de la conexión al servidor ROS como la dirección IP o el puerto entre otros.

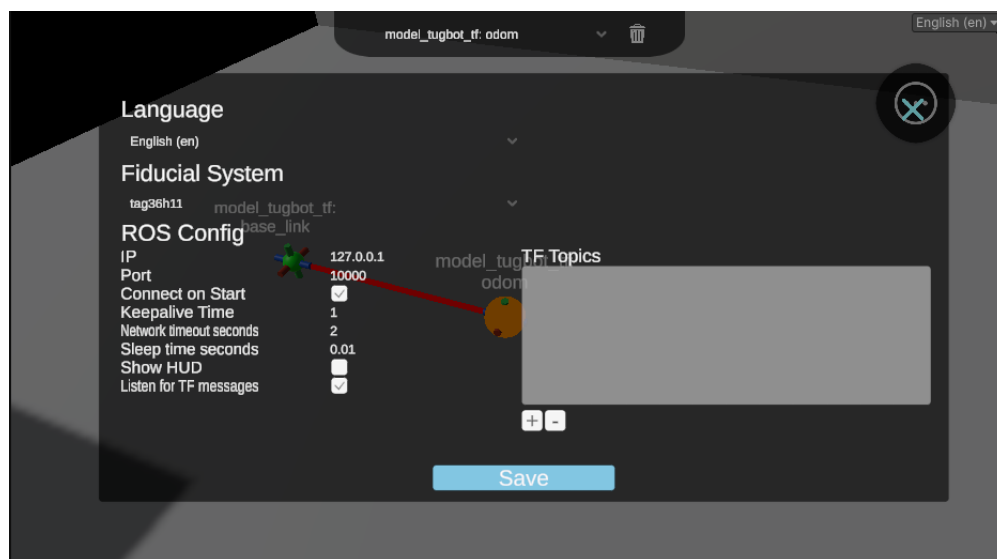


Figura 4.15: Configuración

4.2. Validación de los requisitos

A continuación se muestra el resumen de los requisitos funcionales y no funcionales con su respectivo estado.

Requisito	Estado
RF-001	Cumplido
RF-002	Cumplido
RF-003	Cumplido
RF-004	Cumplido
RF-005	Cumplido
RF-006	Cumplido
RF-007	Cumplido
RF-008	Cumplido
RF-009	Cumplido
RF-010	Cumplido
RF-011	Cumplido
RF-012	Cumplido
RF-013	Cumplido
RF-014	Cumplido

RF-015	Cumplido
RF-016	Cumplido
RF-017	Cumplido
RF-018	Cumplido
RF-019	Cumplido
RF-020	Cumplido
RF-021	Cumplido
RF-022	Cumplido
RF-023	Cumplido
RF-024	Cumplido
RF-025	Cumplido
RF-026	Cumplido
RF-027	Cumplido
RNF-001	NA
RNF-002	NA

Tabla 4.1: Estado de los requisitos funcionales y no funcionales

Como se puede observar, todos los requisitos funcionales han sido cumplimentados en el desarrollo de la aplicación. Por otro lado, los no funcionales no pueden evaluarse de la misma forma que su contraparte funcional, puesto que no se trata de si se ha introducido una funcionalidad concreta o no, sino de la medida en la que se encuentran cubiertos en la aplicación, así pues, no puede darse una respuesta en firme sin que los usuarios finales juzguen con su criterio la aceptación de esos requisitos.

4.3. Resultados de las pruebas

En esta sección se mostrarán la cantidad de pruebas diferentes realizadas, las que han sido un éxito y las que no han sido superadas.

Prueba	Pruebas Realizadas	Pruebas Superadas	Pruebas No Superadas
CP-01			
CP-02			
CP-03			
CP-04			
CP-05	1	1	0

CP-06	1	1	0
CP-07			
CP-08			
CP-09			
CP-10	1	1	0
CP-11	1	1	0
CP-12			
CP-13			
CP-14			
CP-15			
CP-16			
CP-17			
CP-18			
CP-19			
CP-20			
CP-21			
CP-22			
CP-23			
CP-24			
CP-25			
CP-26			
CP-27			
CP-28			
CP-29			
CP-30			
CP-31			
CP-32			
CP-33			
CP-34			
CP-35			
CP-36			

Tabla 4.2: Resultados de las pruebas de integración

4.4. Rendimiento

4.5. Limitaciones encontradas

Durante el desarrollo del proyecto se experimentó con la compilación cruzada del *framework* de ROS para Android, de manera que se pudiera incluir como un *plugin* dentro de la aplicación Unity, sin embargo, la dependencia *rmw_implementation* no es posible compilarla en esta plataforma en el tiempo del desarrollo de la aplicación, ya que necesita de unas modificaciones excesivas en la librería base, que no entran dentro del alcance propuesto en el proyecto.

Para paliar estas restricciones temporales, se optó por investigar la posibilidad de la compilación cruzada de **micro-ROS** igualmente para Android. La compilación y ejecución de este marco de trabajo fue un éxito, sin embargo, durante un análisis y experimentación más exhaustivo, se encontró que no mantiene el grafo ROS necesario para el correcto funcionamiento de la aplicación (visualización de diferentes elementos) por lo que se continuó con la estrategia provisional que había sido introducida en los primeros incrementos del proyecto, haciendo necesaria una dependencia externa (*ROS-TCP-Connector* y nodo propio con capacidad de mantener el grafo ROS) que actúe como soporte para la aplicación y aligerando, por otra parte, los recursos necesarios en el dispositivo móvil.

Como sustituto a **micro-ROS**, se investigó la posibilidad de instalar en un dispositivo Android una máquina virtual ligera Ubuntu empleando *Termux*. A pesar de los problemas iniciales de versionado de Ubuntu, la instalación en el terminal fue un éxito, pudiendo ejecutar una instancia ROS en el *smartphone*. De esta manera, se abre una línea nueva en la que el móvil ejecute tanto ROS como la aplicación de realidad aumentada, desechando la necesidad de dispositivos externos.

Por otro lado, en la funcionalidad de la visualización de nube de puntos, debido a la cantidad de datos procedentes de los sensores que han de procesarse se observa una caída repentina de *frames* de la aplicación. Para paliar estos efectos, se ha realizado un muestreo de los valores proporcionados reduciendo los elementos que deben ser dibujados; además, se ha optado por emplear dos *buffers* de datos, uno para la lectura y otro para la escritura y actualizar la malla visualizada cada 15 *frames* de la aplicación. Aún con todas estas técnicas, debido a que el redibujado de las mallas de puntos es costosa a nivel computacional y que puesto a que la aplicación está destinada a dispositivos móviles y no se pueden emplear elementos de computación paralela como las tarjetas de vídeo, la problemática es todavía presente.

4.6. Resumen de resultados

La aplicación desarrollada cumple con todos los requisitos funcionales propuestos en el proyecto, sin embargo, los no funcionales no pueden ser evaluados de la misma manera, puesto que no se trata de si se ha introducido una funcionalidad concreta o no, sino de la medida en la que se encuentran cubiertos en la aplicación, así pues, no puede darse una respuesta en firme sin que los usuarios finales juzguen con su criterio la aceptación de estos.

Por otro lado, se han encontrado limitaciones en la aplicación que no han podido ser resueltas en el tiempo de desarrollo del proyecto, como la imposibilidad de compilar ROS para Android y la problemática de la visualización de nubes de puntos. A pesar de esto, se ha conseguido desarrollar una aplicación funcional que permite la visualización y control de robots mediante realidad aumentada.

Capítulo 5

Conclusiones y trabajos futuros

5.1. Conclusiones

Durante el desarrollo del proyecto se han afrontado diferentes problemáticas que han requerido de un análisis profundo y de la búsqueda de soluciones que permitan cumplir con los objetivos. Aun así, se han conseguido desarrollar las funcionalidades planteadas en el diseño inicial, y se ha logrado una aplicación funcional que se puede adaptar a cualquier escenario sin configuraciones adicionales – más allá de las imprescindibles – y que permite la visualización y control de robots mediante realidad aumentada, presentando una alternativa real a las aplicaciones existentes basadas en tecnologías tradicionales en las que la interacción es más compleja y se fundamentan en su mayoría en entradas por teclado, uso de ratón y navegación por menús anidados, alejando al usuario del medio real.

5.2. Líneas de trabajo futuras

A pesar de que la aplicación desarrollada cumple con los objetivos planteados, se han encontrado limitaciones que no han podido ser resueltas en tiempo y forma en el desarrollo del proyecto, pero que podrían ser abordadas en un futuro para mejorar la experiencia de usuario y la funcionalidad de la aplicación.

Una de estas limitaciones es la dificultad de compilar ROS para Android, lo que obliga a emplear dependencias externas que rompen con el planteamiento original de que la aplicación fuese completamente autónoma. Durante la investigación se encontró que aunque era realmente complejo y requería de un tiempo del que no se disponía, las librerías problemáticas en la compilación cruzada podían ser modificadas de modo que se eliminasen los errores y, lo que es más importante, estuviesen preparadas para ejecutarse sobre Android. Esto permitiría que la aplicación fuese completamente autónoma y no dependiese de servicios externos para su funcionamiento, lo que podría mejorar la experiencia de usuario.

Otra solución plausible para la compilación de ROS para Android que podría ser abora-

dada en un futuro sería la utilización de máquinas virtuales que ejecuten ROS y los servicios necesarios en el dispositivo Android, de modo que la aplicación se conectase a estas máquinas virtuales y no dependiese de servicios externos. En este sentido, se experimentó con la utilización de Termux como emulador del terminal en Android y se consiguió instalar una máquina virtual Ubuntu en la que se instaló ROS, superando la barrera de la compilación cruzada y permitiendo ejecutar los servicios directamente en el dispositivo móvil. El siguiente paso sería conectar la aplicación al servicio de ROS ejecutado en la máquina virtual y comprobar que la comunicación es correcta y que la aplicación funciona como se espera.

Por otro lado, la visualización de nubes de puntos es una funcionalidad que no se ha podido implementar de manera satisfactoria debido a la complejidad de la misma y a las limitaciones de los dispositivos móviles. Para solventar esta problemática, se podrían emplear técnicas de computación paralela que permitan procesar y renderizar las nubes de puntos de manera más eficiente, aprovechando al máximo los recursos del dispositivo móvil. Además, se podrían explorar alternativas de visualización que reduzcan la carga computacional, como la simplificación de las nubes de puntos o el uso de representaciones aun más ligeras.

Finalmente, y como punto adicional, aunque no sería imprescindible para el funcionamiento de la aplicación, se podrían realizar pruebas de entorno con usuarios finales para evaluar su experiencia y la aceptación de la aplicación, y de este modo, obtener datos valiosos que permitan mejorar la aplicación y adaptarla mejor a las necesidades de los usuarios. Cabe destacar que el proyecto ha sido desarrollado empleando entornos simulados, por lo que la experiencia real en este momento podría diferir de la obtenida en los entornos de prueba.

Bibliografía

- Adekoya, Omotoye Shamsudeen, Antonio Sgorbissa y Carmine Tommaso Recchiuto (2025). *HORUS: A Mixed Reality Interface for Managing Teams of Mobile Robots*. arXiv: 2506.02622 [cs.R0]. URL: <https://arxiv.org/abs/2506.02622>.
- Ai, Letian, Peter Kazanzides y Ehsan Azimi (abr. de 2024). «Mixed reality based teleoperation and visualization of surgical robotics». English. En: *Healthcare Technology Letters* 11.2-3. Copyright - © 2024. This work is published under <http://creativecommons.org/licenses/by-nc-nd/4.0/> (the "License"). Notwithstanding the ProQuest Terms and Conditions, you may use this content in accordance with the terms of the License; Última actualización - 2024-08-08, págs. 179-188. URL: <https://www.proquest.com/scholarly-journals/mixed-reality-based-teleoperation-visualization/docview/3090589709/se-2>.
- App ROS2 Connect Mobile - App Store* — *apps.apple.com* (s.f.). <https://apps.apple.com/es/app/ros2-connect-mobile/id6744851782>.
- Calderón-Sesmero, Raúl et al. (2025). «Human–Robot Interaction and Tracking System Based on Mixed Reality Disassembly Tasks». En: *Robotics* 14.8. ISSN: 2218-6581. DOI: [10.3390/robotics14080106](https://www.mdpi.com/2218-6581/14/8/106). URL: <https://www.mdpi.com/2218-6581/14/8/106>.
- Chen, P. et al. (2018). «KnowEdu: A System to Construct Knowledge Graph for Education». En: *IEEE Access* 6, págs. 31553-31563. DOI: [10.1109/ACCESS.2018.2839607](https://doi.org/10.1109/ACCESS.2018.2839607).
- Dessi, Danilo et al. (2020). *AI-KG: an Automatically Generated Knowledge Graph of Artificial Intelligence*.
- Foxglove Documentation* — *docs.foxglove.dev* (s.f.). <https://docs.foxglove.dev/docs>.
- Friske, Michael D. (2024). *Integration of Augmented Reality and Mobile Robot Indoor SLAM for Enhanced Spatial Awareness*. arXiv: 2409.01915 [cs.R0]. URL: <https://arxiv.org/abs/2409.01915>.
- Ji, Shaoxiong et al. (2020). *A Survey on Knowledge Graphs: Representation, Acquisition and Applications*. arXiv: 2002.00388 [cs.CL].
- Katherine Scott, Tully Foote (s.f.). *2023 ROS Metrics Report*. <http://download.ros.org/downloads/metrics/metrics-report-2024-01.pdf>.
- Maeda, Ryuichi et al. (2024). «Sensor Pods and ROS2-TMS for Construction for Cyber-Physical System at Earthwork Sites». En: *2024 IEEE International Symposium on Safety Security Rescue Robotics (SSRR)*, págs. 58-63. DOI: [10.1109/SSRR62954.2024.10770031](https://doi.org/10.1109/SSRR62954.2024.10770031).

- Natephra, Worawan y Ali Motamedi (mayo de 2019). «Live Data Visualization of IoT Sensors Using Augmented Reality (AR) and BIM». En: *Proceedings of the 36th International Symposium on Automation and Robotics in Construction (ISARC)*. Ed. por Mohamed Al-Hussein. Banff, Canada: International Association for Automation y Robotics in Construction (IAARC), págs. 632-638. ISBN: 978-952-69524-0-6. DOI: [10.22260/ISARC2019/0084](https://doi.org/10.22260/ISARC2019/0084).
- Neves, João, Diogo Serrario y J. N. Pires (2018). «Application of mixed reality in robot manipulator programming». English. En: *The Industrial Robot* 45.6. Nombre - Microsoft Corp; Copyright - © Emerald Publishing Limited 2018; Última actualización - 2024-12-02, págs. 784-793. URL: <https://www.proquest.com/scholarly-journals/application-mixed-reality-robot-manipulator/docview/2151235465/se-2>.
- Qin, Yuehua, Han Cao y Leyi Xue (ago. de 2020). «Research and Application of Knowledge Graph in Teaching: Take the database course as an example». En: *Journal of Physics: Conference Series* 1607, pág. 012127. DOI: [10.1088/1742-6596/1607/1/012127](https://doi.org/10.1088/1742-6596/1607/1/012127). URL: <https://doi.org/10.1088/1742-6596/1607/1/012127>.
- Regal, Frank et al. (2024). «AR-STAR: An Augmented Reality Tool for Online Modification of Robot Point Cloud Data». En: *Companion of the 2024 ACM/IEEE International Conference on Human-Robot Interaction. HRI '24*. Boulder, CO, USA: Association for Computing Machinery, págs. 872-876. ISBN: 9798400703232. DOI: [10.1145/3610978.3640571](https://doi.org/10.1145/3610978.3640571). URL: <https://doi.org/10.1145/3610978.3640571>.
- Richards, Lachlan et al. (feb. de 2025). «Replayable Augmented Reality Visualization for Robot Fault Diagnosis: A Comparative Study». En: *International Journal of Social Robotics* 17, págs. 2133-2145. DOI: [10.1007/s12369-025-01219-4](https://doi.org/10.1007/s12369-025-01219-4).
- Robot Steering App* - App Store — [apps.apple.com](https://apps.apple.com/gh/app/robot-steering/id6749877170) (s.f.). <https://apps.apple.com/gh/app/robot-steering/id6749877170>.
- ROS-Mobile* - ROS Wiki — [wiki.ros.org](https://wiki.ros.org/{R}{O}{S}-{M}obile) (s.f.). <https://wiki.ros.org/{R}{O}{S}-{M}obile>.
- ROS: Why ROS?* — [ros.org](https://www.ros.org/blog/why-ros/) (s.f.). <https://www.ros.org/blog/why-ros/>.
- Rosales, Jonathan, Sourabh Deshpande y Sam Anand (2021). «IIoT based Augmented Reality for Factory Data Collection and Visualization». En: *Procedia Manufacturing* 53. 49th SME North American Manufacturing Research Conference (NAMRC 49, 2021), págs. 618-627. ISSN: 2351-9789. DOI: <https://doi.org/10.1016/j.promfg.2021.06.062>. URL: <https://www.sciencedirect.com/science/article/pii/S235197892100072X>.
- RViz User Guide & ROS 2 Documentation: Kilted documentation* — docs.ros.org (s.f.). <https://docs.ros.org/en/kilted/{T}utorials/{I}ntermediate/{R}{V}iz/{R}{V}iz-{U}ser-{G}uide/{R}{V}iz-{U}ser-{G}uide.html>.
- Schmidt, Bernard et al. (abr. de 2022). «Augmented Reality Approach for a User Interface in a Robotic Production System». En: ISBN: 9781643682686. DOI: [10.3233/ATDE220143](https://doi.org/10.3233/ATDE220143).
- webviz* — webviz.io (s.f.). <https://webviz.io/>.

- Yang, Zhewen et al. (2025). «Remote Human-Robot Collaboration in XR». En: *Proceedings of the 26th International Workshop on Mobile Computing Systems and Applications*. Hot-Mobile '25. La Quinta, CA, USA: Association for Computing Machinery, pág. 131. ISBN: 9798400714030. DOI: [10.1145/3708468.3715687](https://doi.org/10.1145/3708468.3715687). URL: <https://doi.org/10.1145/3708468.3715687>.
- Zea, Antonio y Uwe D. Hanebeck (2021). «iviz: A ROS visualization app for mobile devices». En: *Software Impacts* 8, pág. 100057. ISSN: 2665-9638. DOI: <https://doi.org/10.1016/j.simpa.2021.100057>. URL: <https://www.sciencedirect.com/science/article/pii/S2665963821000051>.

Apéndice A

Ejemplos en L^AT_EX

Este primer apéndice presenta ejemplos en L^AT_EX de cómo incluir referencias, citas bibliográficas, figuras, tablas o código. Este apéndice se deberá eliminar de la memoria antes de entregar el trabajo.

A.1. Referencias, citas y bibliografía

A.1.1. Referencias

Para poder referenciar un elemento dentro de la memoria hay que marcarlo con una etiqueta (`\label{[id]}`) que lo identifique inequívocamente. Es habitual utilizar identificadores representativos, por ejemplo, para marcar la introducción podemos utilizar una etiqueta como `\label{chap:introduccion}`

Una vez marcado el elemento (capítulo, sección, figura, tabla, ...) en L^AT_EX, utilizaremos el comando `ref` indicando a qué etiqueta queremos referenciar (`\ref{[id]}`). Por ejemplo, de esta manera podemos referenciar al capítulo de la introducción con 1. Podemos acompañarlo de un texto como “...como se vió en el capítulo 1...” o bien utilizar el comando `\autoref{[i]}`, que incluiría el tipo del elemento y aparecería en el texto como Capítulo 1, o incluso referenciarlo por el nombre del elemento con `\nameref{[id]}`, lo que se mostraría como Introducción.

A.1.2. Bibliografía

Para añadir una cita bibliográfica al documento tendremos que asegurarnos que en el fichero de bibliografía (bibliografía.bib) se encuentre la entrada bibliográfica correspondiente. Una vez que tengamos nuestra entrada bibliográfica utilizaremos el comando de cita de L^AT_EX indicando el identificador de la referencia bibliográfica, por ejemplo: `\cite{aikg}`, que se se visualizaría como Dessi et al. 2020.

Cuando queramos citar más de un trabajo en un mismo punto del documento se deberán

añadir a la cita separados por comas, por ejemplo: `\cite{chen_2018, Qin_2020}`, que se visualizaría como Chen et al. 2018; Qin, Cao y Xue 2020. En estos casos es habitual añadir las citas en orden cronológico de más antigua a más reciente.

Tal y como está configurada esta plantilla, por defecto el comando `\cite` realizará una cita textual (`\citet` si se activa el paquete `natbib` en lugar de `biblatex`), es decir, la cita forma parte del texto. Este tipo de citas se suelen realizar en frases como:

“Algunos ejemplos de grafos de conocimiento se presentan en Ji et al. 2020”

Cuando la cita no forma parte del texto si no que se utiliza para reforzar una afirmación realizada en una frase, se debe utilizar una cita entre paréntesis (o corchetes). Para ello se usa el comando `\parencite` en `biblatex` o `\citep` en `natbib`, por ejemplo:

“Existe una gran variedad de aplicaciones de los grafos de conocimiento (Ji et al. 2020).”

A.2. Figuras

Para añadir una figura al documento utilizaremos el entorno `figure`, indicando la posición que debe tener dicha figura en el documento (h: here, t: top, b: botom; p: page), se recomienda en lo posible evitar de “!” que ignora todos los ajustes de los parámetros. El orden en el que se indiquen cada una de las opciones se tendrá en cuenta para colocar la figura, es decir si se indicase el orden `[htbp]` la figura primero se intentará colocar en el lugar que ocupa en el documento, si no se puede se intentará colocar al inicio (top) de la página, en caso que tampoco sea posible se intentará colocar al final (bottom) de la página y, por último en caso que no sea posible ninguna de las anteriores se colocará al inicio de una página nueva.



Figura A.1: Esta figura tiene una descripción al pie muy larga, por lo que añadiremos un título breve utilizando para ello los corchetes tras el comando `\caption`. La etiqueta de la figura (label) se incluirá al inicio del flotante de la figura para que cualquier referencia cruzada (ref) a la misma lleve al inicio del flotante.

A.3. Tablas

Para añadir una tabla se utilizará el entorno `table`, indicando al inicio de la tabla el título de la misma utilizando el `caption`. Un ejemplo puede verse en la Tabla A.1.

Tabla A.1: Esta tabla presenta un ejemplo con tres columnas y formato formal.

Model	Accuracy	Precision	Recall	F1-score	AUC
Modelo 1	0,33	0,75	0,72	0,42	0,21
Modelo 2	0,01	0,63	0,60	0,50	0,10
Modelo 3	0,03	0,93	0,33	0,04	0,42

A.4. Código

Para añadir código en la memoria utilizaremos el paquete `listing`, que permite mostrar código formateado en diversos lenguajes (Java, Python, C, ...).

```

1 import numpy as np
2
3 def incmatrix(genl1,genl2):
4     m = len(genl1)
5     n = len(genl2)
6     M = None #to become the incidence matrix
7     VT = np.zeros((n*m,1), int) #dummy variable
8
9     #compute the bitwise xor matrix
10    M1 = bitxormatrix(genl1)
11    M2 = np.triu(bitxormatrix(genl2),1)
12
13    for i in range(m-1):
14        for j in range(i+1, m):
15            [r,c] = np.where(M2 == M1[i,j])
16            for k in range(len(r)):
17                VT[(i)*n + r[k]] = 1;
18                VT[(i)*n + c[k]] = 1;
19                VT[(j)*n + r[k]] = 1;
20                VT[(j)*n + c[k]] = 1;
21
22            if M is None:
23                M = np.copy(VT)
24            else:
25                M = np.concatenate((M, VT), 1)
26
27            VT = np.zeros((n*m,1), int)
28
29    return M

```

Código A.1: Título del fragmento de código

A.5. Listas

Como en cualquier procesador de textos debemos diferenciar dos tipos de listas. Las listas no numeradas (o de viñetas) se definirán mediante entornos `itemize`.

- Elemento no numerado
- Elemento no numerado
- Elemento no numerado

Las listas numeradas se definirán mediante entornos `enumerate`. En ambos casos, cada elemento de la lista se genera utilizando el comando `item`.

1. Elemento 1

Con el comando `\` podemos insertar saltos de línea sin cambiar de párrafo.

2. Elemento 2. Con el comando `\textbf` podemos **enfaticar con negrita** un texto y con el comando `\textit` podemos *enfaticar con itálica* un texto.

3. Elemento 3

A.6. Ecuaciones

Cuando una ecuación va a ser referenciada desde el texto, en principio más de una vez, será necesario nombrar o numerar dicha ecuación (ver Ecuación A.1). Para añadir una ecuación numerada utilizaremos el entorno `equation`.

$$\begin{pmatrix} w_1, & w_2, & \dots, & w_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} + b = 0 \rightarrow \mathbf{w}^t \mathbf{x} + b = 0 \quad (\text{A.1})$$

Cuando la ecuación no va a ser referenciada desde el texto, podemos utilizar la versión de entorno no numerada `equation*`

$$\frac{1}{2} \|\mathbf{w}\| = \frac{1}{2} \sqrt{\sum w_i^2}$$

Es también posible añadir ecuaciones en línea con el texto, para ello se debe incluir la expresión matemática en L^AT_EX encerrada entre símbolos de dólar. Por ejemplo, la ecuación anterior se podría representar también en línea: $\frac{1}{2} \|\mathbf{w}\| = \frac{1}{2} \sqrt{\sum w_i^2}$.